BLM2502 The Theory of Computation

Spring 2022

Lecturer: Dr. Oğuz Altun

Email: oaltun@yildiz.edu.tr

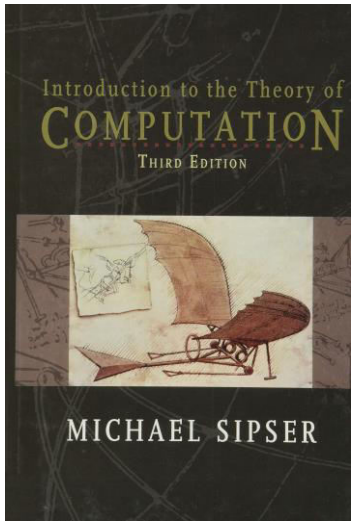
Office: D036

Office Hours: Tuesday 12:00-13:00

- **Course: BLM 2502 The theory of Computation**
  - **Tuesday 09:00-12:00**
  - **ECTS 3 credits**

BLM2502 Theory of Computation

- **Book**
  - **Michael Sipser, Introduction to the Theory of Computation (3E), Thomson**
  - **No handouts, its your responsibily to take notes.**



  - **Lectures from the author:**

https://ocw.mit.edu/courses/mathematics/18-404j-theory-of-computation-fall-2020/

» **Course Goals:**

» To introduce fundamental theoretical concepts for the computational complexity of algorithmic problems, and also to introduce basic techniques used for lexical analysis and syntax parsing in programming languages.

» **Main Topics Covered:**

» Finite Automata

- Deterministic and nondeterministic finite automata
- Regular expressions

» Pushdown Automata

- Context-free languages and grammars
- Parsing

» Turing Machines

- Turing recognizable languages
- Decidability
- Time and Space complexity
- NP-completeness

- **Grading:**
  - Homework assignments: 15%
  - Pop-up quizzes (~8 quizzes): 15%
  - Midterm: 30%
  - Final:40%

BLM2502 Theory of Computation

Spring 2017

BLM2502 Theory of Computation

- **Book**
  - **Michael Sipser, Introduction to the Theory of Computation (3E), Thomson**
  - **No handouts, its your responsibily to take notes.**

BLM2502 Theory of Computation

- In this Theory of Computation course we will try to answer the following questions:
- What are the mathematical properties of computer hardware and software?
- What is a computation and what is an algorithm? Can we give rigorous mathematical definitions of these notions?
- What are the limitations of computers? Can "everything" be computed? (As we will see, the answer to this question is "no".)
- **Purpose of the Theory of Computation:**

  Develop formal mathematical models of computation that reflect real-world computers.

BLM2502 Theory of Computation

- Complexity Theory
- The main question asked in this area is “What makes some problems computationally hard and other problems easy?”
- Informally, a problem is called “easy”, if it is efficiently solvable. Examples of “easy” problems are
  - Sorting a sequence of, e.g, 1,000,000 numbers,
  - Searching for a name in a telephone directory
  - Computing the fastest way to drive from Esenler to Beşiktaş.

BLM2502 Theory of Computation

- Complexity Theory
- On the other hand, a problem is called “hard”, if it cannot be solved efficiently, or if we don’t know whether it can be solved efficiently. Examples of “hard” problems are
  - Time table scheduling for all courses
  - Factoring a 300-digit integer into its prime factors
  - Computing a layout for chips in VLSI.
- **Central Question in Complexity Theory:**

  Classify problems according to their degree of “difficulty”. Give a rigorous proof that problems that seem to be “hard” are really “hard”.

BLM2502 Theory of Computation

- Computability Theory
- In the 1930's, scientists discovered that some of the fundamental mathematical problems cannot be solved by a "computer". (computers were invented in 1940s). For example: "Is an arbitrary mathematical statement true or false?"
- To attack such a problem, we need formal definitions of the notions of
  - computer
  - algorithm
  - computation

BLM2502 Theory of Computation

- Computability Theory
- The theoretical models that were proposed in order to understand solvable and unsolvable problems led to the development of real computers.
- Central Question in Computability Theory:

  Classify problems as being solvable or unsolvable.

BLM2502 Theory of Computation

- Automata Theory
- Automata Theory deals with definitions and properties of different types of “computation models”. Examples :
  - Finite Automata. These are used in text processing, compilers, and hardware design.
  - Context-Free Grammars. These are used to define programming languages and in Artificial Intelligence.
  - Turing Machines. These form a simple abstract model of a “real” computer, such as your PC at home.
- Central Question in Automata Theory:

  Do these models have the same power, or can one model solve more problems than the other?

BLM2502 Theory of Computation

- Set Theory
- A set is a collection of well-defined objects.
  - The set of **natural numbers** is **N** = {1, 2, 3, . . .}.
  - The set of **integers** is **Z** = {. . . ,−3,−2,−1, 0, 1, 2, 3, . . .}.
  - The set of **rational numbers** is **Q** = {m/n : m $\in$ **Z**, n $\in$ **Z**, n $\neq$ 0}.
    - What is irrational numbers?
  - The set of **real numbers** is denoted by **R**.
  - If A and B are sets, then A is a **subset** of B, written as A $\subseteq$ B, if every element of A is also an element of B.
  - If B is a set, then the **power set** *P*(B) of B is defined to be the set of all subsets of B:
    - *P*(B) = {A : A $\subseteq$ B}.
    - Observe $\phi \in$ P(B) and B $\in$ P(B).

BLM2502 Theory of Computation

- Set Theory
  - If A and B are two sets, then
    - their union is defined as
      - $A \cup B = \{x : x \in A \text{ or } x \in B\}$,
    - their intersection is defined as
      - $A \cap B = \{x : x \in A \text{ and } x \in B\}$,
    - their difference is defined as
      - $A \setminus B = \{x : x \in A \text{ and } x \notin B\}$,
    - the Cartesian product of A and B is defined as
      - $A \times B = \{(x, y) : x \in A \text{ and } y \in B\}$,
    - the complement of A is defined as
      - $\bar{A} = \{x : x \notin A\}$.

BLM2502 Theory of Computation

[, 0, 00, 11, 1, 10, 01

» Set Theory

- A **binary relation** on two sets A and B is a subset of A × B.
- A **function** f from A to B, denoted by f : A → B, is a binary relation *R*, having the property that for each element a ∈ A, there is exactly one ordered pair in *R*, whose first component is a. We will also say that f(a) = b, or f maps a to b, or the image of a under f is b. The set A is called the **domain** of f, and the set

  {b ∈ B : there is an a ∈ A with f(a) = b} is called the **range** of f.
- A binary relation R ⊆ A × A is an **equivalence relation**, if it satisfies the following three conditions:
  - R is **reflexive**: For every element in a ∈ A, we have (a, a) ∈ R.
  - R is **symmetric**: For all a and b in A, if (a, b) ∈ R, then (b, a) ∈ R.
  - R is **transitive**: For all a, b, and c in A, if (a, b) ∈ R and (b, c) ∈ R, then also (a, c) ∈ R.

BLM2502 Theory of Computation

- Boolean Logic
- The Boolean values are 1 and 0, that represent true and false, respectively. The basic Boolean operations:
- **negation** (or NOT), represented by $\neg$,
- **conjunction** (or AND), represented by Λ,
- **disjunction** (or OR), represented by V,
- **exclusive-or** (or XOR), represented by $\oplus$,
- **equivalence**, represented by $\leftrightarrow$ or $\Leftrightarrow$,
- **implication**, represented by $\rightarrow$ or $\Rightarrow$. ,

BLM2502 Theory of Computation

- Truth Table (0=false, 1=true)

NOT	AND	OR	XOR	equivalence	implication
$\neg 0 = 1$	$0 \wedge 0 = 0$	$0 \vee 0 = 0$	$0 \oplus 0 = 0$	$0 \Leftrightarrow 0 = 1$	$0 \Rightarrow 0 = 1$
$\neg 1 = 0$	$0 \wedge 1 = 0$	$0 \vee 1 = 1$	$0 \oplus 1 = 1$	$0 \Leftrightarrow 1 = 0$	$0 \Rightarrow 1 = 1$
	$1 \wedge 0 = 0$	$1 \vee 0 = 1$	$1 \oplus 0 = 1$	$1 \Leftrightarrow 0 = 0$	$1 \Rightarrow 0 = 0$
	$1 \wedge 1 = 1$	$1 \vee 1 = 1$	$1 \oplus 1 = 0$	$1 \Leftrightarrow 1 = 1$	$1 \Rightarrow 1 = 1$

- Implication (if … then …)
  - antecedent (condition)->consequence (promise)
- E.g.
  - p: “you take out the trash”.
  - q:”you get a dollar”
  - p=>q is false only if you take out the trash but don’t get a dolar.

BLM2502 Theory of Computation

- Proof Techniques
- In mathematics, a **theorem** is a statement that is true. A **proof** is a sequence of mathematical statements that form an argument to show that a theorem is true.
  - **Axioms**: assumptions about the underlying mathematical structures
  - **Hypotheses**: a supposition or proposed explanation made based on limited evidence as a starting point for further investigation.
  - **Theorem**: described above
  - **Lemmas**: previously proved theorems
  - **Corollaries**: Special cases of theorem
- There is no specified way of producing a proof, but there are some generic strategies that could be of help.

BLM2502 Theory of Computation

- Proof Techniques
- Tips:
  - **Read** and completely understand the statement of the theorem to be proved. Most often this is the hardest part. **Rewrite** the statement in your own words.
  - Sometimes, theorems contain theorems inside them. For example, “Property A if and only if property B”, requires showing **two statements**:
    - (a) If property A is true, then property B is true (A→B).
    - (b) If property B is true, then property A is true (B→ A).
  - Another example is the theorem “Set A equals set B.” To prove this, we need to prove that $A \subseteq B$ and $B \subseteq A$. That is, we need to show that each element of set A is in set B, and that each element of set B is in set A.

BLM2502 Theory of Computation

- Proof Techniques
- Tips:
  - Try to **work out a few simple cases** of the theorem just to get a grip on it (i.e., crack a few simple cases first).
  - Try to **write down the proof** once you have it. This is to ensure the correctness of your proof. Often, mistakes are found at the time of writing.
  - Finding proofs takes time, we do not come prewired to produce proofs. **Be patient**, think, express and write clearly and try to be precise as much as possible.

BLM2502 Theory of Computation

- Proof Techniques
  - Direct Proofs or Constructive Proofs or Proof by Construction
  - Nonconstructive Proofs
  - Proofs by Contradiction
  - Proofs by Induction
  - Pigeon Principle

BLM2502 Theory of Computation

- Proof Techniques - **Direct Proofs**
- As the name suggests, in a direct proof of a theorem, we just approach the theorem directly.
- **Theorem:** If n is an odd positive integer, then n^2 is odd as well.
- **Proof:** An odd positive integer n can be written as:

  $n = 2k + 1$, for some integer $k \geq 0$. Then:

  - $n^2 = (2k + 1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1$.
  - Since $2(2k^2 + 2k)$ is even, and "even plus one is odd", we can conclude that
  - n^2 is odd.

BLM2502 Theory of Computation

- Proof Techniques - Constructive Proofs

  (Proof By Construction)

- Many theorems state that a particular type of object exists
- One way to prove is to find a way to construct one such object
- This technique is called proof by construction
- **Theorem:** There exists a rational number p which can be expressed as a^b, with a and b both irrational.

A *constructive* proof of the above theorem on irrational powers of irrationals would give an actual example, such as:

$$a = \sqrt{2}\,, \quad b = \log_2 9\,, \quad a^b = 3\,.$$

The square root of 2 is irrational, and 3 is rational. $\log_2 9$ is also irrational: if it were equal to $\frac{m}{n}$, then, by the properties of logarithms, 9^n would be equal to 2^m, but the former is odd, and the latter is even.

BLM2502 Theory of Computation

- Proof Techniques - **Proof by Contradiction**
- The proof by contradiction is grounded in the fact that **any proposition must be either true or false, but not both true and false at the same time**.
- One common way to prove a theorem is to assume that the theorem is false, and then show that this assumption leads to an obviously false consequence (also called a contradiction)
- This type of reasoning is used frequently in everyday life.

BLM2502 Theory of Computation

- Proof Techniques - **Proof by Contradiction**
- Let us define a number is rational if it can be expressed as p/q where p and q are integers; if it cannot, then the number is called irrational.
- **Theorem:** $\sqrt{2}$ (the square root of 2) is irrational.
- **Proof:** Assume that $\sqrt{2}$ is rational. Then, it can be written as p/q for some positive integers p and q such that **p and q does not have a common factor**.
- Then, we have $p^2/q^2 = 2$, or $2q^2 = p^2$
- (continued in next page)

BLM2502 Theory of Computation

- Proof Techniques - **Proof by Contradiction**
- Since $2q^2$ is an even number, p^2 is also an even number
- This implies that **p is an even number** (why?)
- So, $p = 2r$ for some integer r, and so, $2q^2 = p^2 = (2r)^2 = 4r^2$
- This implies $2r^2 = q^2$
- So, **q is an even number**
- Something wrong happens!.. We now have:
- "p and q does not have common factor"
- AND
- "**p and q have common factor**"
- **This is a contradiction**
- Thus, the assumption is wrong, so that $\sqrt{2}$ is irrational

BLM2502 Theory of Computation

- Proof Techniques - **Proof by Induction**
- For each positive integer n, let P(n) be a mathematical statement that depends on n. Assume we wish to prove that P(n) is true for all positive integers n. A proof by induction of such a statement is carried out as follows:
- **Basis**: Prove that P(1) is true.
- **Induction step**: Prove that for all $n \geq 1$, the following holds: If P(n) is true, then P(n + 1) is also true.
- In the induction step, we choose an arbitrary integer n 1 and assume that P(n) is true; this is called the induction hypothesis. Then we prove that P(n + 1) is also true.

BLM2502 Theory of Computation

- Proof Techniques - **Proof by Induction**
- **Theorem:** For all positive integers n, we have

  $1 + 2 + 3 + \ldots + n = n(n + 1)/2$

- **Proof:** Start with the basis of the induction. If n = 1, then the left-hand side is equal to 1, and so is the right-hand side. So the theorem is true for n = 1.
- For the induction step, let $n \geq 1$ and assume that the theorem is true for n, i.e., assume that

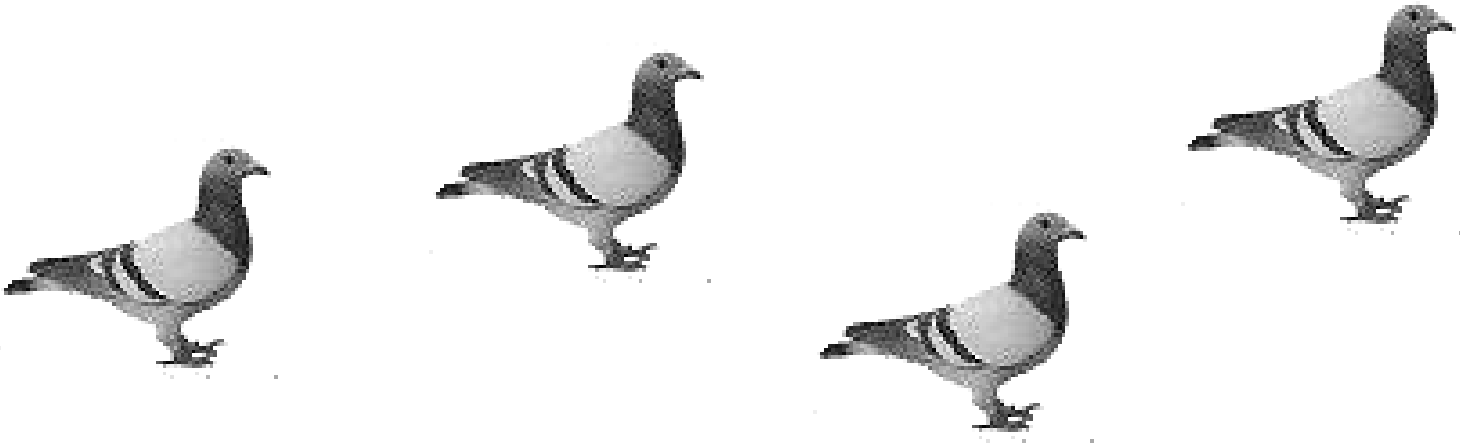
  $1 + 2 + 3 + \ldots + n = n(n + 1)/2$

- We have to prove that the theorem is true for n + 1

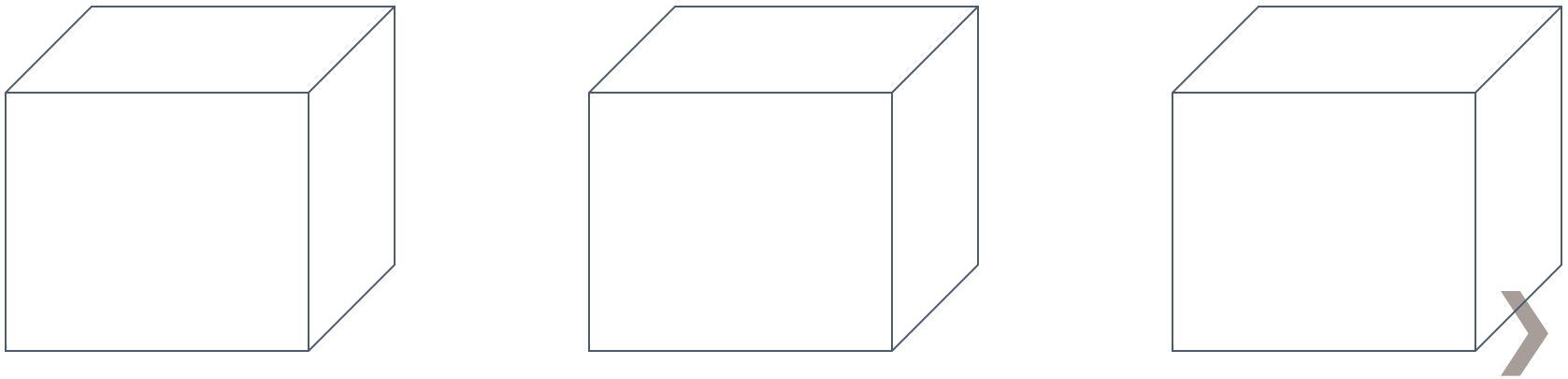
BLM2502 Theory of Computation

- Proof Techniques – **Pigeonhole Principle**
- If n + 1 or more objects are placed into n boxes, then there is at least one box containing two or more objects.
- In other words, if A and B are two sets such that |A| > |B|, then there is no one-to-one function from A to B.

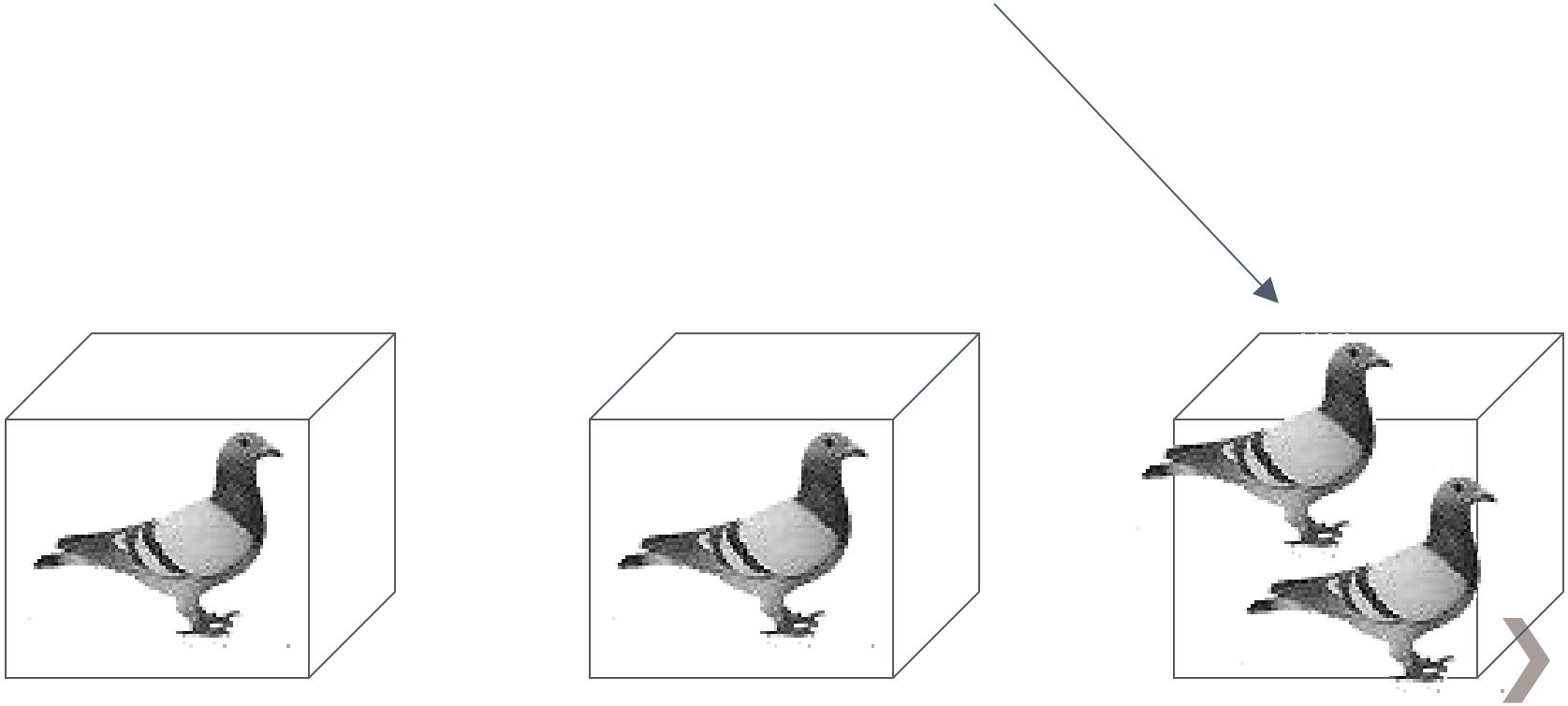
4 pigeons



3 pigeonholes



A pigeonhole must
contain at least two pigeons



- Proof techniques: Pigeonhole Principle.
- Example: Prove that if seven distinct numbers are selected from{1,2,...,11},then some two of these numbers sum to 12
- All numbers from 1 to 11 can be put into following **6 pigeonholes**: {1,11},{2,10},{3,9},{4,8},{5,7},{6}.
- We select **7** distinct numbers (**pigeons**). First 6 pigeon can be put to different pigeonholes. But after that we have to put to an existing pigeonhole. The pigeonhole of 6 can hold only one pigeon ☺.

BLM2502 Theory of Computation

- Proof Techniques – Pigeonhole Principle
- **Theorem:** Let n be a positive integer. Every sequence of $n^2 + 1$ distinct natural numbers contains a subsequence of length n + 1 that is either increasing or decreasing.
- **Proof:** For example consider the sequence

  (8,**11**,**9**,1,4,**6**,12,10,**5**,7)

of $10 = 3^2 + 1$ numbers. This sequence contains a decreasing subsequence of length 4 = 3 + 1, shown. There are other subsequences of length 4, too.

Proof: Let $a_1, a_2, \ldots, a_{n^2+1}$ be a sequence of $n^2 + 1$ distinct real numbers. Associate an ordered pair with each term of the sequence, namely, associate (i_k, d_k) to the term a_k, where i_k is the length of the longest increasing subsequence starting at a_k, and d_k is the length of the longest decreasing subsequence starting at a_k.

Suppose that there are no increasing or decreasing subsequences of length $n + 1$. Then i_k and d_k are both positive integers less than or equal to n, for $k = 1, 2, \ldots, n^2 + 1$. Hence, by the product rule there are n^2 possible ordered pairs for (i_k, d_k). By the pigeonhole principle, two of these $n^2 + 1$ ordered pairs are equal. In other words, there exist terms a_s and a_t, with $s < t$ such that $i_s = i_t$ and $d_s = d_t$. We will show that this is impossible. Because the terms of the sequence are distinct, either $a_s < a_t$ or $a_s > a_t$. If $a_s < a_t$, then, because $i_s = i_t$, an increasing subsequence of length $i_t + 1$ can be built starting at a_s, by taking a_s followed by an increasing subsequence of length i_t beginning at a_t. This is a contradiction. Similarly, if $a_s > a_t$, the same reasoning shows that d_s must be greater than d_t, which is a contradiction. ◁

Sequence=(8,11,9,1,4,6,12,10,5,7)

$$a_1 = 8, (i_1 = 3, d_1 = 3)$$
$$a_2 = 11, (i_2 = 2, d_2 = 4)$$
$$a_3 = 9, (i = 2, d_2 = 3)$$
$$\ldots$$

If there are no sequences of length $n + 1$, there are only n^2 pairs. Then since we have $n^2 + 1$ distinct numbers. At least 2 pairs are equal by pigeonhole principle. But this is not possible as numbers are **distinct**.

BLM2502 Theory of Computation

Spring 2017

BLM2502 Theory of Computation

- **Course Outline**
- Week Content
- 1 Introduction to Course
- 2 Computability Theory, Complexity Theory, Automata Theory, Set Theory, Relations, Proofs, Pigeonhole Principle
- **3 Regular Languages**
- 4 **Finite Automata**
- 5 Deterministic and Nondeterministic Finite Automata
- 6 Epsilon Transition, Equivalence of Automata
- 7 Pumping Theorem
-
- 9 Context Free Grammars
- 10 Parse Tree, Ambiguity,
- 11 Pumping Theorem
- 13 Turing Machines, Recognition and Computation, Church-Turing Hypothesis
- 14 Turing Machines, Recognition and Computation, Church-Turing Hypothesis
- 15 Review

Regular Expressions

BLM2502 Theory of Computation

Regular Languages

- Keywords / Definitions:
  - *Alphabet*: A finite nonempty set of symbols. The members of the alphabet are the *symbols* of the alphabet. We generally use capital Greek letters Σ and Γ to designate alphabets. The following are a few examples of alphabets.

    $\Sigma_1 = \{0,1\}$

    $\Sigma_2 = \{a,b,c,d,e,f,g.h,i,j,k,l,m,n,o,p,q,r,s,t,u,v,w,x,y,z\}$

    $\Gamma = \{0, 1, x, y, z\}$
  - A string over an alphabet is a finite sequence of symbols from that alphabet, usually written next to one another and not separated by commas. If $\Sigma_1 = \{0,1\}$, then 01101 is a string over Σ_1. If $\Sigma_2 = \{a, b, c, \ldots, z\}$, then abracadabra is a string over Σ_2.

BLM2502 Theory of Computation

Regular Expressions

- Keywords / Definitions:
  - If w is a string over Σ, the *length* of w, written |w|, is the number of symbols that it contains. The string of length zero is called the *empty string* and is written as ε (or λ). The empty string plays the role similiar to 0 in a number system.
  - If w has length n, we can write

    $w = w_1 w_2 \ldots w_n$ where each $w_i \in \Sigma$.

    The reverse of w, written as w^R, is the string obtained by writing w in the opposite order (i.e., $w_n w_{n-1} \ldots w_1$).

    String z is a *substring* of w if z appears consecutively within w. For example, *cad* is a substring of *abracadabra*

BLM2502 Theory of Computation

Regular Expressions

- Keywords / Definitions:
  - If we have string x of length m and string y of length n, the *concatenation* of x and y, written xy, is the string obtained by appending y to the end of x, as in $x_1 \ldots x_m y_1 \ldots y_n$. To concatenate a string with itself many times we use the superscript notation:

    $x^3 = xxx$,

    $x^n = xx\ldots.x$ (n times)

    To indicate all possible recurrencies, a special superscript (in fact a special operator) is used:

    * (kleene star)
  - *Language* : A set of strings (finite or infinite?) over an alphabet. Languages are used to describe computation problems.

BLM2502 Theory of Computation

Regular Expressions

» Keywords / Definitions:

> *Regular Languages*: A subset of all languages. There is no way to define regular set. However, it is possible to say whether a set is regular or not.

> Best way is using *Finite Automata*. If some finite automata recognizes the language, then the language is regular.

> Regular (set) operations are used to build up regular sets:

> Union, concatenation, and kleene star operations are as follows.

> *Union*: $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$.

> *Concatenation*: $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$.

> *Star*: $A^* = \{x_1 x_2 \ldots x_k \mid k > 0 \text{ and each } x_i \in A\}$.

BLM2502 Theory of Computation

Regular Expressions

- Keywords / Definitions:
  - Class of regular languages is closed under **union**
  - Class of regular languages is closed under **intersection**
  - Class of regular languages is closed under **complement**
  - Class of regular languages is closed under **concenation**
  - Class of regular languages is closed under (kleene) **star**

BLM2502 Theory of Computation

Alphabet: $= \{a, b\}$

Strings:

a

ab

$abba$

$aaabbbaabab$

$u = ab$

$v = bbbaaa$

$w = abba$

BLM2502 Theory of Computation

Decimal numbers

Alphabet: $\Sigma = \{0,1,2,\ldots,9\}$

Strings:

102345 567463386 …

Binary numbers

Alphabet: $\Sigma = \{0,1\}$

Strings:

100010001 101101111 …

BLM2502 Theory of Computation

Unary numbers

Alphabet: $\Sigma = \{1\}$

Strings:

$1 \quad 11 \quad 111111 \quad \ldots$

Decimal equivalent:

$1,2,6\ldots$

BLM2502 Theory of Computation

- Examples on String Operations

  $w = a_1a_2\ldots a_n$, $v = b_1b_2\ldots b_m$ $a,b \in \{x, y\}$

  eg, w = xyyxy, v = xxxyyy

  Concenation:

  $wv = a_1a_2\ldots a_nb_1b_2\ldots b_n$

  eg, xyyxyxxxyyy

  Reverse:

  $w^R = a_na_{n-1}\ldots a_1$

  eg, yxyyx

BLM2502 Theory of Computation

- Examples on String Operations

  $w = a_1a_2\ldots a_n$, $v = b_1b_2\ldots b_m$ $a,b \in \{x, y\}$

  Length:

  $|w| = n$, $|v| = m$

  eg, $|yxyyx| = 5$; $|xxxyyy| = 6$

  $|wv| = |w| + |v|$

  eg, $|yxyyxxxxyyy| = 11$ (recall example above)

  Empty String:

  $|\varepsilon| = 0$

BLM2502 Theory of Computation

» Examples on String Operations

$w = a_1a_2\ldots a_n$, $v = b_1b_2\ldots b_m$ $a,b \in \{x, y\}$

Substring:

$a_1, a_2, \ldots, a_n$ (each symbol of the string are its substrings)

$a_1, a_1a_2, a_1a_2a_3\ldots$ (these are prefix substrings)

$a_2, a_2a_3, a_2a_3a_4, \ldots$

$a_n, a_{n-1}a_n, a_{n-2}a_{n-1}a_n, \ldots$(these are suffix substrings)

special cases:

ε is prefix and suffix of each string

any string is prefix and suffix of itself

BLM2502 Theory of Computation

» Examples on String Operations

Special cases:

ε is prefix and suffix of each string

any string is prefix and suffix of itself

String: abba

Prefix	Suffix
ε	abba
a	bba
ab	ba
abb	a
abba	ε

Observe that:
εw = wε = w

BLM2502 Theory of Computation

- Examples on String Operations

  $w = a_1a_2\ldots a_n$, $v = b_1b_2\ldots b_m$ $a,b \in \{x, y\}$

  Self Concenation:

  $ww = w^2$, $www = w^3$, etc

  $w^0 = \varepsilon$

  eg, w = abba;

  $(abba)^0 = \varepsilon$

  $(abba)^1 = abba$

  $(abba)^2 = abbaabba$

  $(abba)^3 = abbaabbaabba$

  Kleene Star:

  $w^* = \{w^0, w^1, w^2, w^3, \ldots\}$

BLM2502 Theory of Computation

- The * operation:

  The set of all possible strings from the alphabet

  $\Sigma = \{a, b\}$

  $\Sigma^* = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, aab, \ldots\}$

- The + operation:

  The set of all possible strings from the alphabet excluding ε

  $\Sigma = \{a, b\}$

  $\Sigma^+ = \Sigma^* - \{\varepsilon\} = \{a, b, aa, ab, ba, bb, aaa, aab, \ldots\}$

BLM2502 Theory of Computation

- Language:

  A Language over an alphabet $\Sigma = \{a, b\}$

  is a subset of $\Sigma^* = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, aab, \ldots\}$

  Examples:

  $L_1 = \{\varepsilon\}$

  $L_2 = \{a, b, aa, ab, ba, bb\}$

  $L_3 = \{\varepsilon, a, aa, aaa, aaaa, \ldots\}$

  $L_4 = \{a^n b^n, n \geq 0\} = \{\varepsilon, ab, aabb, aaabbbb, \ldots\}$

  $\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

  PRIME_NUMBERS $= \{x \mid x \in \Sigma^*$ and x is prime $\}$

  $= \{2, 3, 5, 7, 11, \ldots\}$

  EVEN_NUMBERS $= \{x \mid x \in \Sigma^*$ and x is even$\} = \{0, 2, 4, \ldots\}$

  ODD_NUMBERS $= \{x \mid x \in \Sigma^*$ and x is odd$\} = \{1, 3, 5, \ldots\}$

BLM2502 Theory of Computation

- Unary Addition

  Alphabet $\Sigma = \{1, +, =\}$

  ADDITION = $\{x + y = z: x = 1^m, y=1^n, z = 1^k; k = m + n\}$

  $111 + 11 = 11111 \in$ ADDITION

  $111 + 111 = 1111 \notin$ ADDITION

- Squaring

  Alphabet $\Sigma = \{1, \#\}$

  SQUARES= $\{x \# y : x = 1^m, y=1^n, n = m^2\}$

  $111 \# 111111111 \in$ SQUARES

  $1111 \# 11111111 \notin$ SQUARES

BLM2502 Theory of Computation

» Operations On Languages

Set Operations: Regular sets are closed under union, intersection negation and complement.

A = {a, ab, aaaa}

B = {ab, bb}

$A \cup B$ = {a, ab, bb, aaaa}

$A \cap B$ = {ab}

A - B = {a, bb, aaaa}

$\bar{A} = \Sigma^* - A = \{a, ab, aaaa\}^{-} = \{\varepsilon, aa, ba, bb, aba, \ldots\}$

Note that :

$\emptyset = \{\} \neq \{\varepsilon\}$

$|\emptyset| = |\{\}| = 0 \neq |\{\varepsilon\}|$

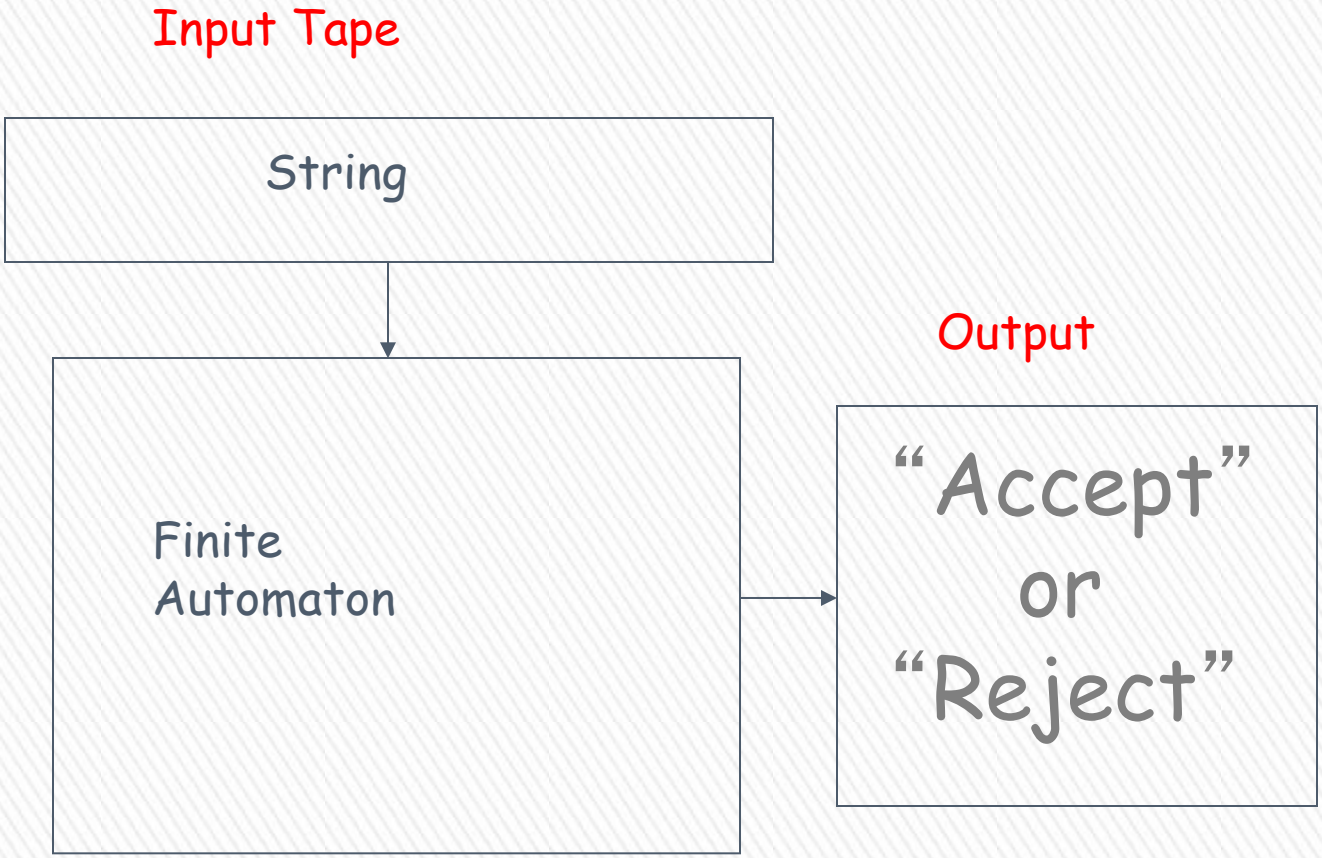
$|\{\varepsilon\}| = 1$ * This is set size

$|\varepsilon| = 0$ * This is string length

Finite Automata

BLM2502 Theory of Computation

»



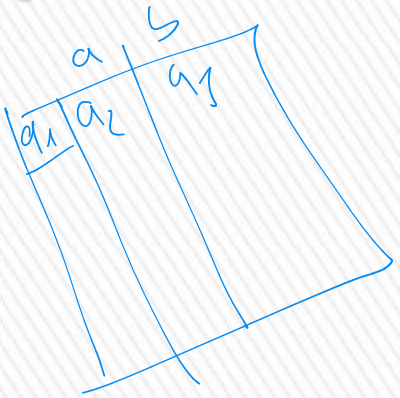
BLM2502 Theory of Computation

Finite Automata

» A finite automaton is a 5-tuple:

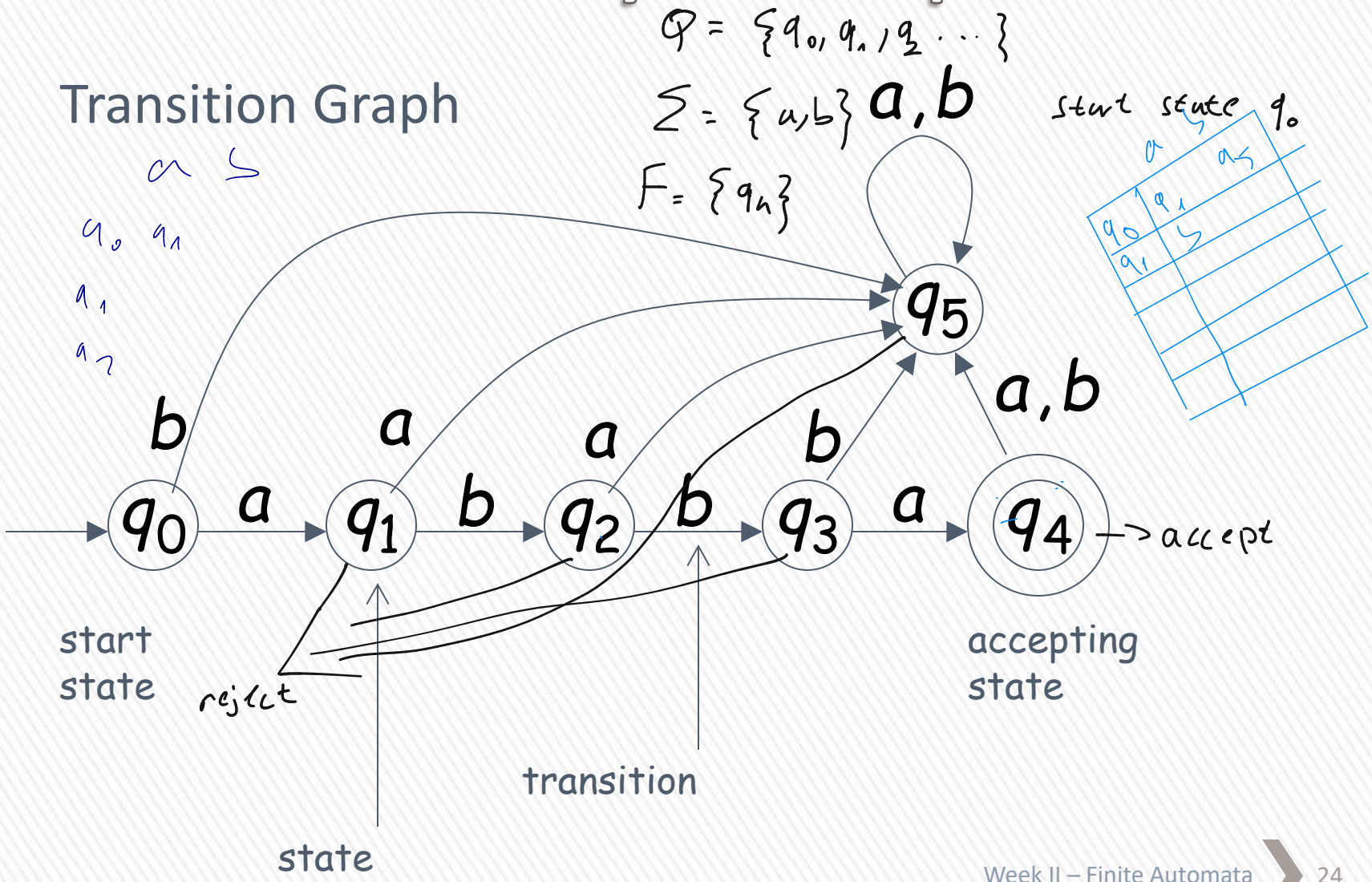
(Q, Σ, δ, q_0, F) where;

1. Q is a finite set called the ***states,***
2. Σ is a finite set called the ***alphabet,***
3. δ: Q x Σ → Q is the ***transition function,***
4. $q_0 \in Q$ is the ***start state,*** and
5. $F \subseteq Q$ is the ***set** of **accept states.***



BLM2502 Theory of Computation

Transition Graph



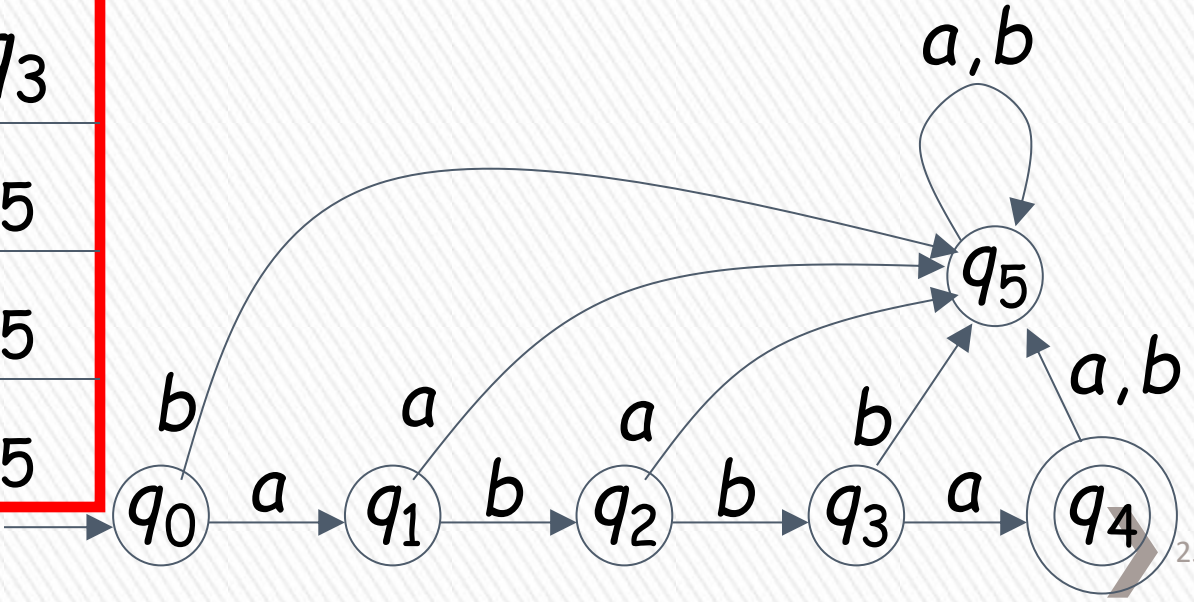
BLM2502 Theory of Computation

symbols

states

δ	a	b
q_0	q_1	q_5
q_1	q_5	q_2
q_2	q_5	q_3
q_3	q_4	q_5
q_4	q_5	q_5
q_5	q_5	q_5

Transition Table



BLM2502 Theory of Computation

- You can think of the transition function as being the "program" of the finite automaton M. This function tells us what M can do in "one step":
  - Let r be a state of Q and let a be a symbol of the alphabet Σ. If the finite automaton M is in state r and reads the symbol a, then it switches from state r to state $\delta(r, a)$. (In fact, $\delta(r, a)$ may be equal to r.)
- Example:
  - A = {w : w is a binary string containing an odd number of 1s}.

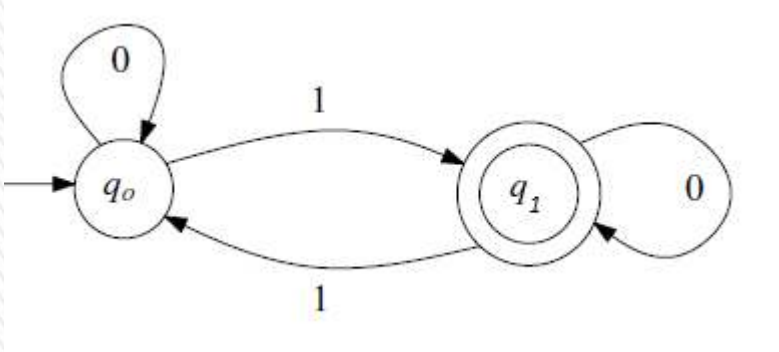
BLM2502 Theory of Computation

Design:

- The finite automaton reads the input string w from left to right and keeps track of the number of 1s it has seen. After having read the entire string w, it checks whether this number is odd (in which case w is accepted) or even (in which case w is rejected).
- Using this approach, the finite automaton needs a state for every integer $i \geq 0$, indicating that the number of 1s read so far is equal to i.

BLM2502 Theory of Computation

- Design – Continued
- However, this design is not feasible since FA have finite number of states.
- ???
- A better, and correct approach, is to keep track of whether the number of 1s read so far is even or odd.



BLM2502 Theory of Computation

$M = (Q, \Sigma, \delta, q_0, F)$

$Q = \{q_0, q_1\}$

$\Sigma = \{0, 1\}$ (trivial from the problem)

$q_0 \in Q$

$F = \{q_1\}$

δ:

$(q_0, 0) \rightarrow q_0$

$(q_0, 1) \rightarrow q_1$

$(q_1, 0) \rightarrow q_1$

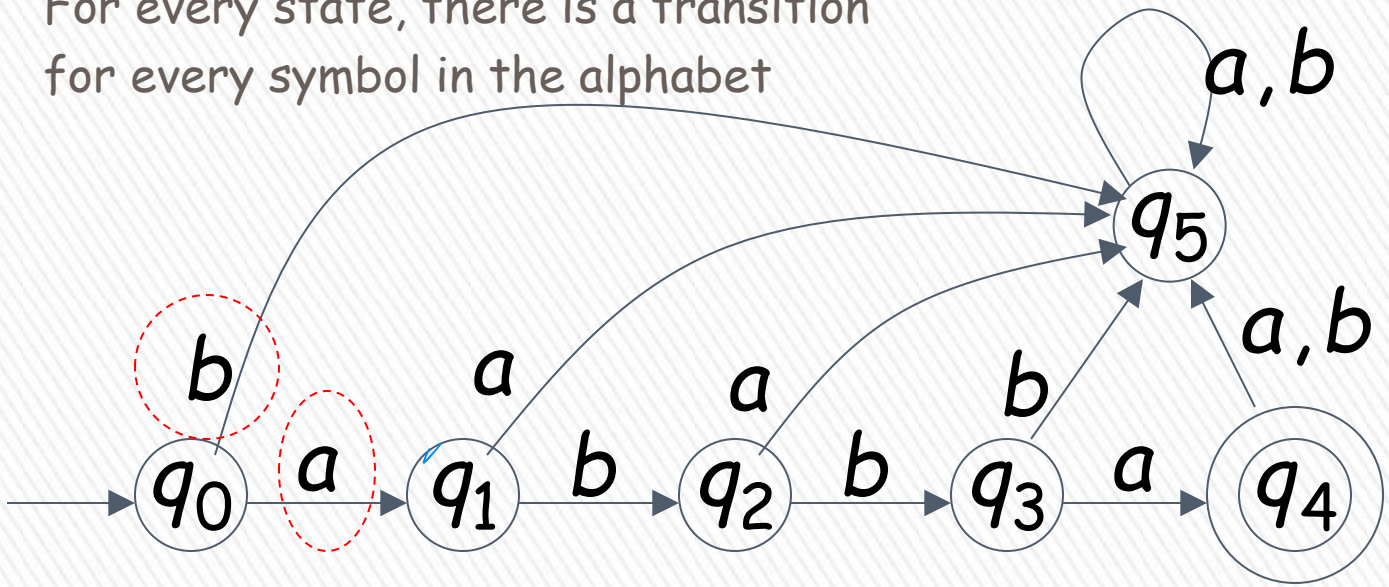
$(q_1, 1) \rightarrow q_0$

δ:	0	1
q_0	q_0	q_1
q_1	q_1	q_0

BLM2502 Theory of Computation

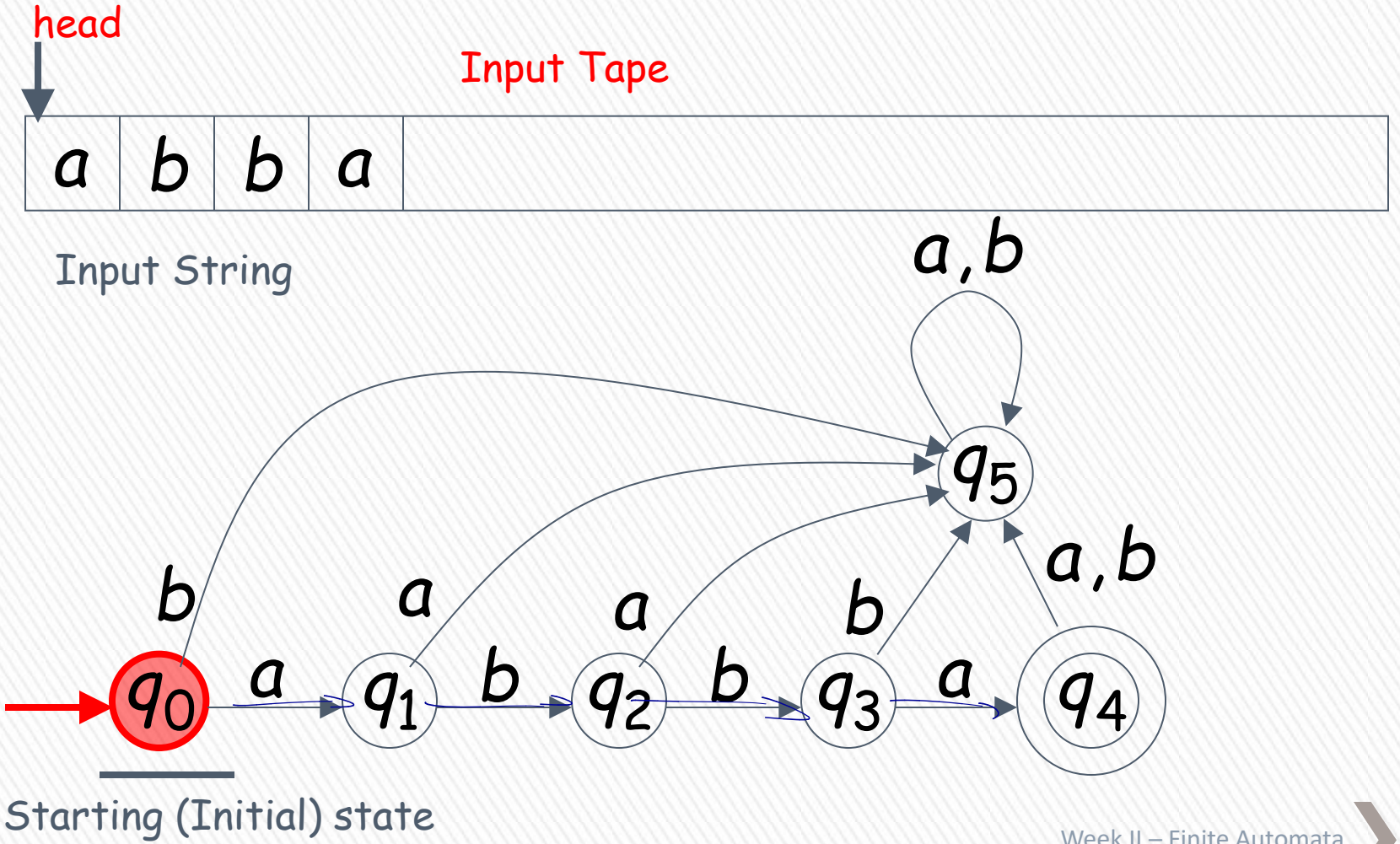
» DFA - Deterministic Finite Automata

For every state, there is a transition
for every symbol in the alphabet

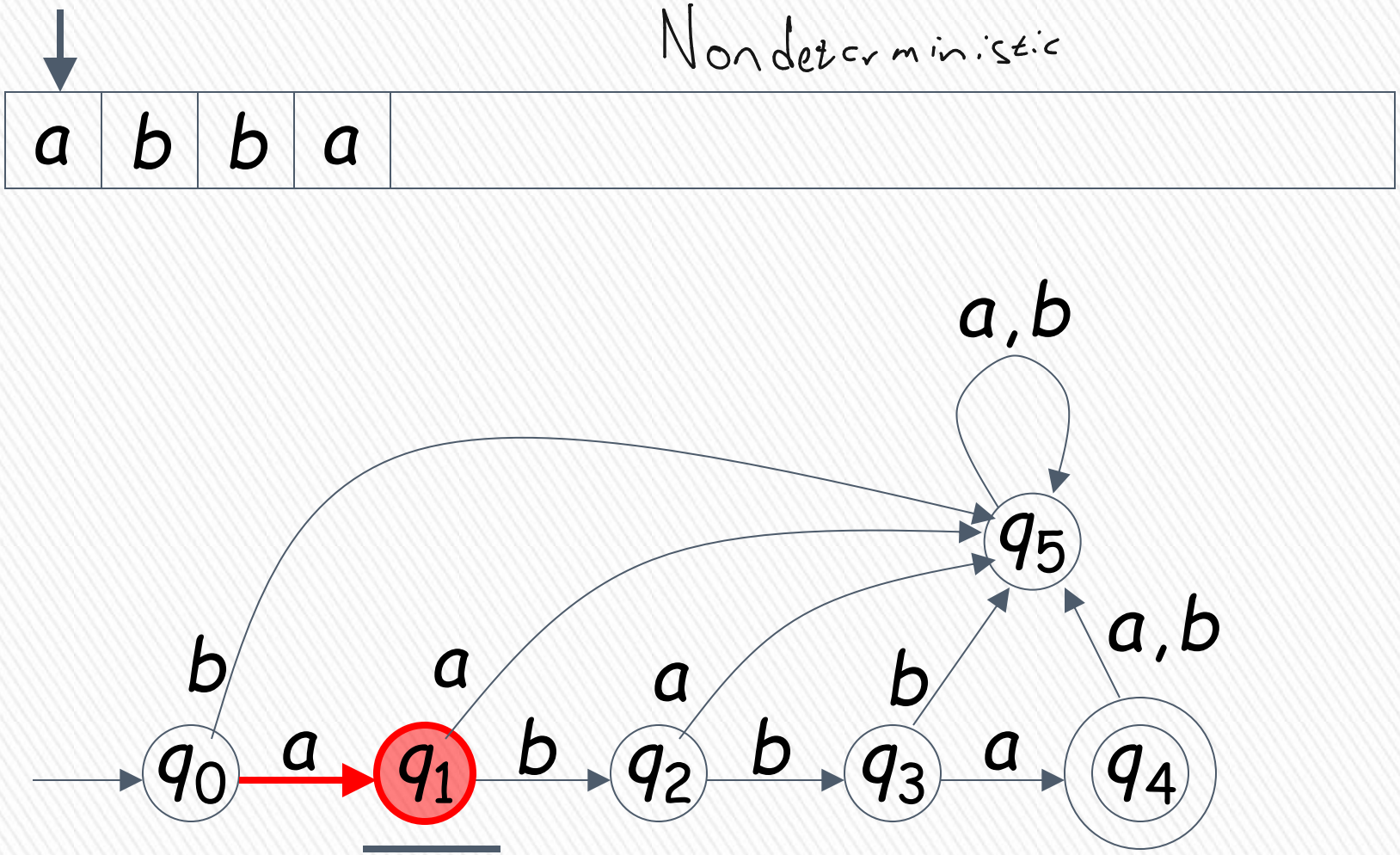


Alphabet Σ = {a, b}

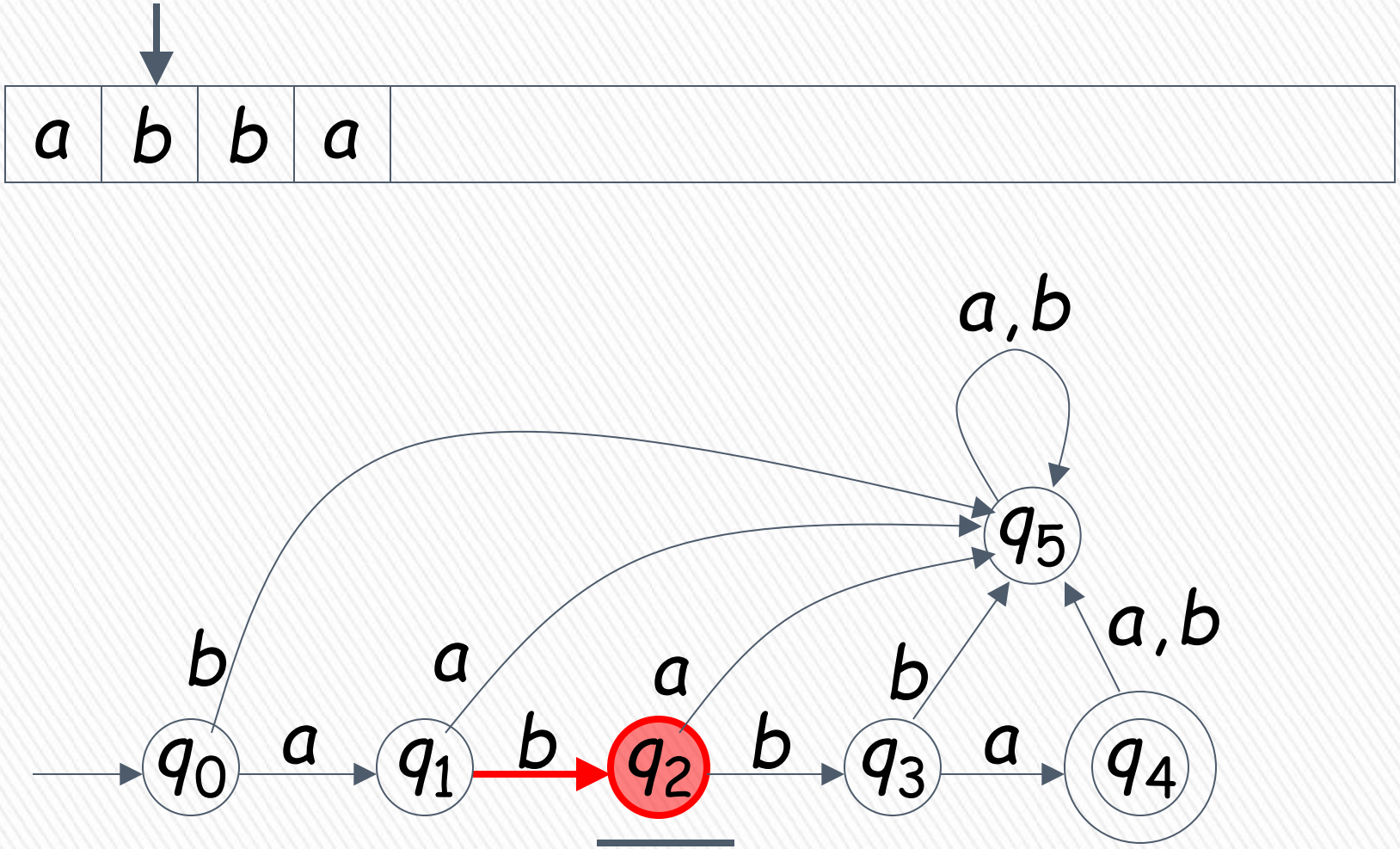
BLM2502 Theory of Computation



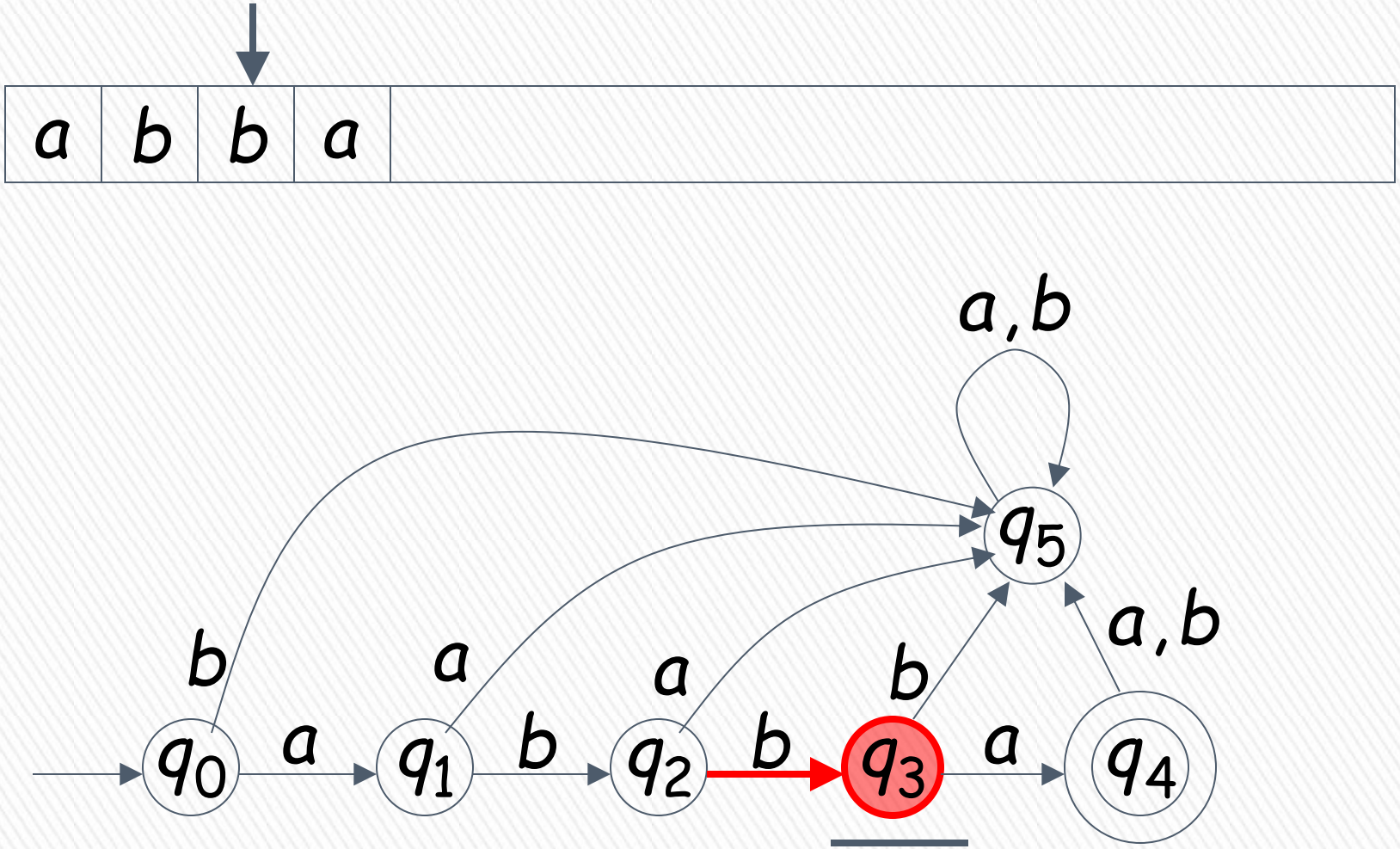
BLM2502 Theory of Computation

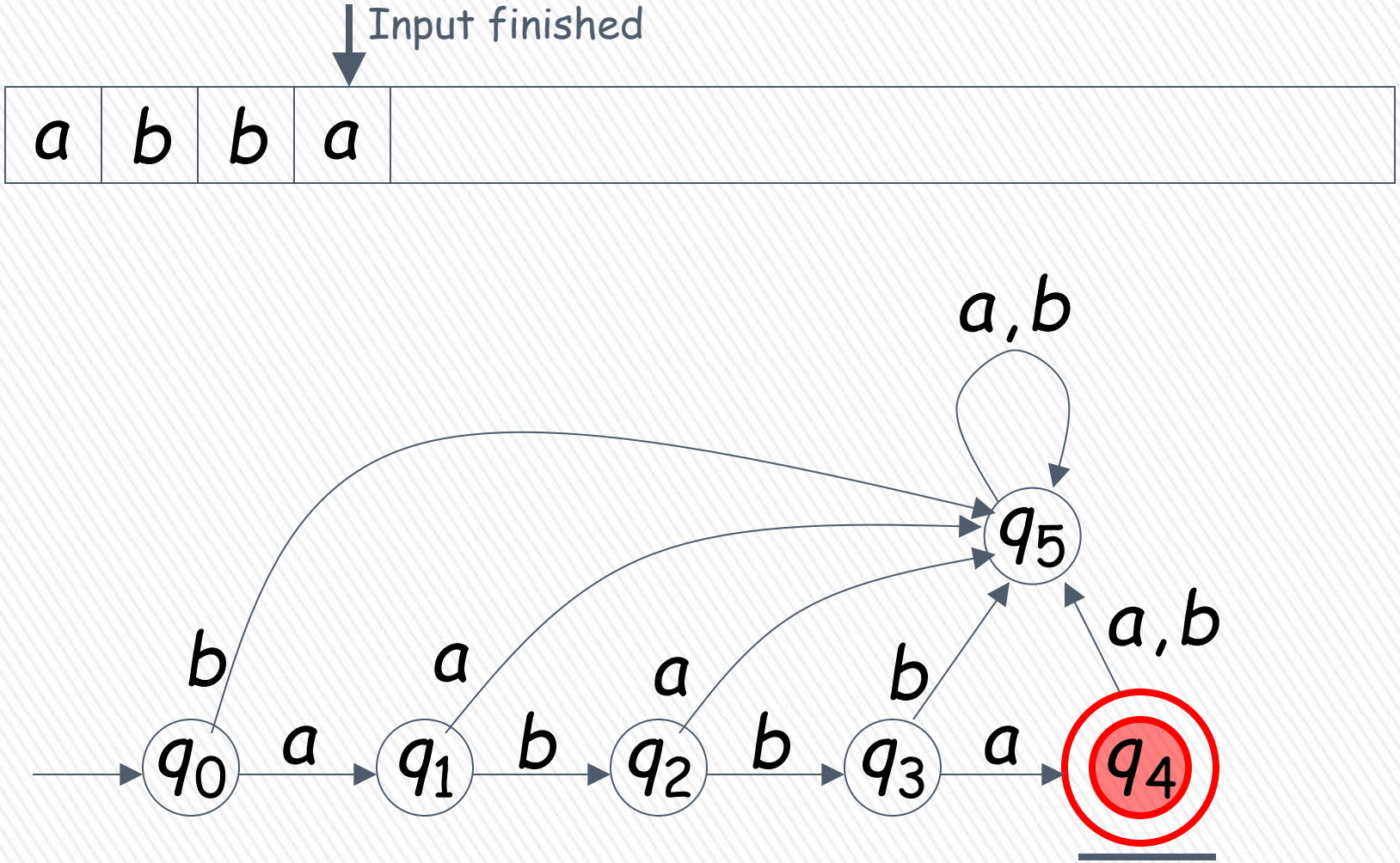


BLM2502 Theory of Computation



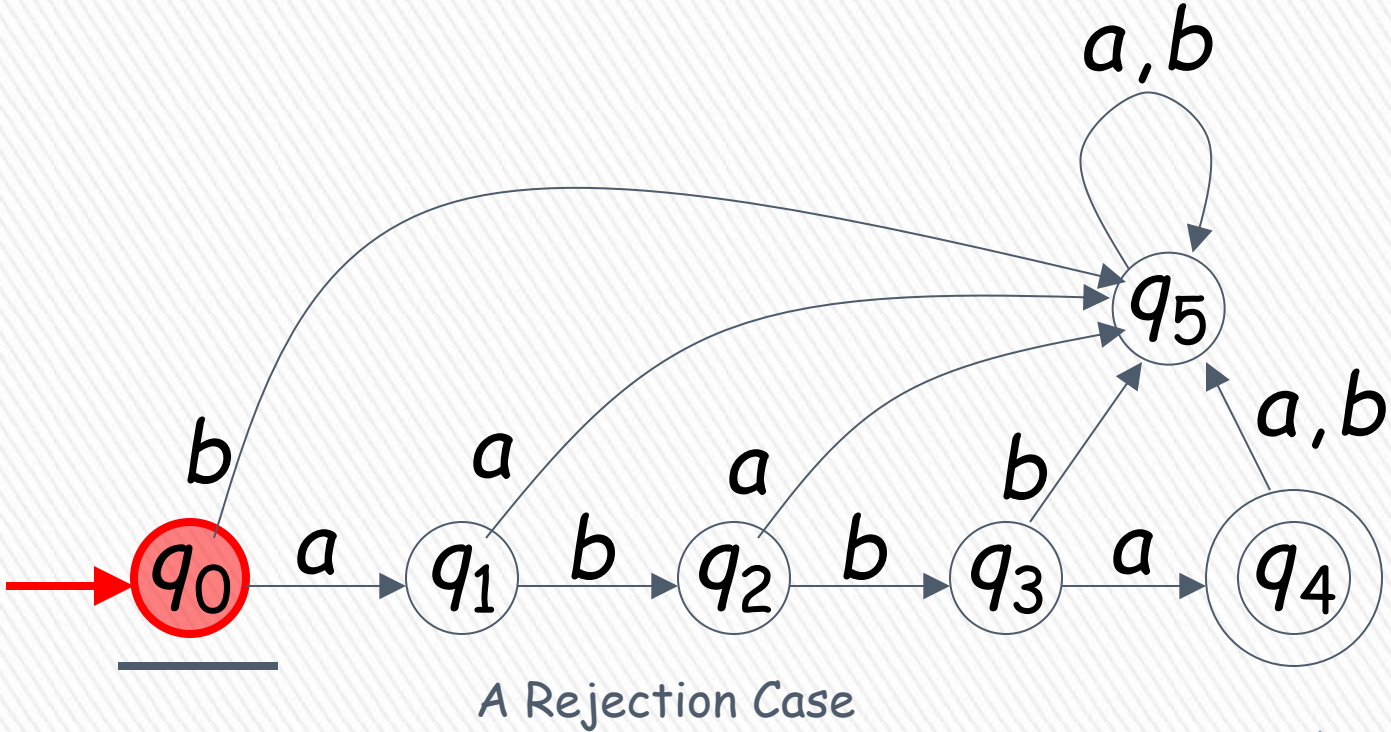
BLM2502 Theory of Computation




Input finished
a b b a
a,b
q5
b
a
a
b
a,b
q0
a
q1
b
q2
b
q3
a
q4
accept

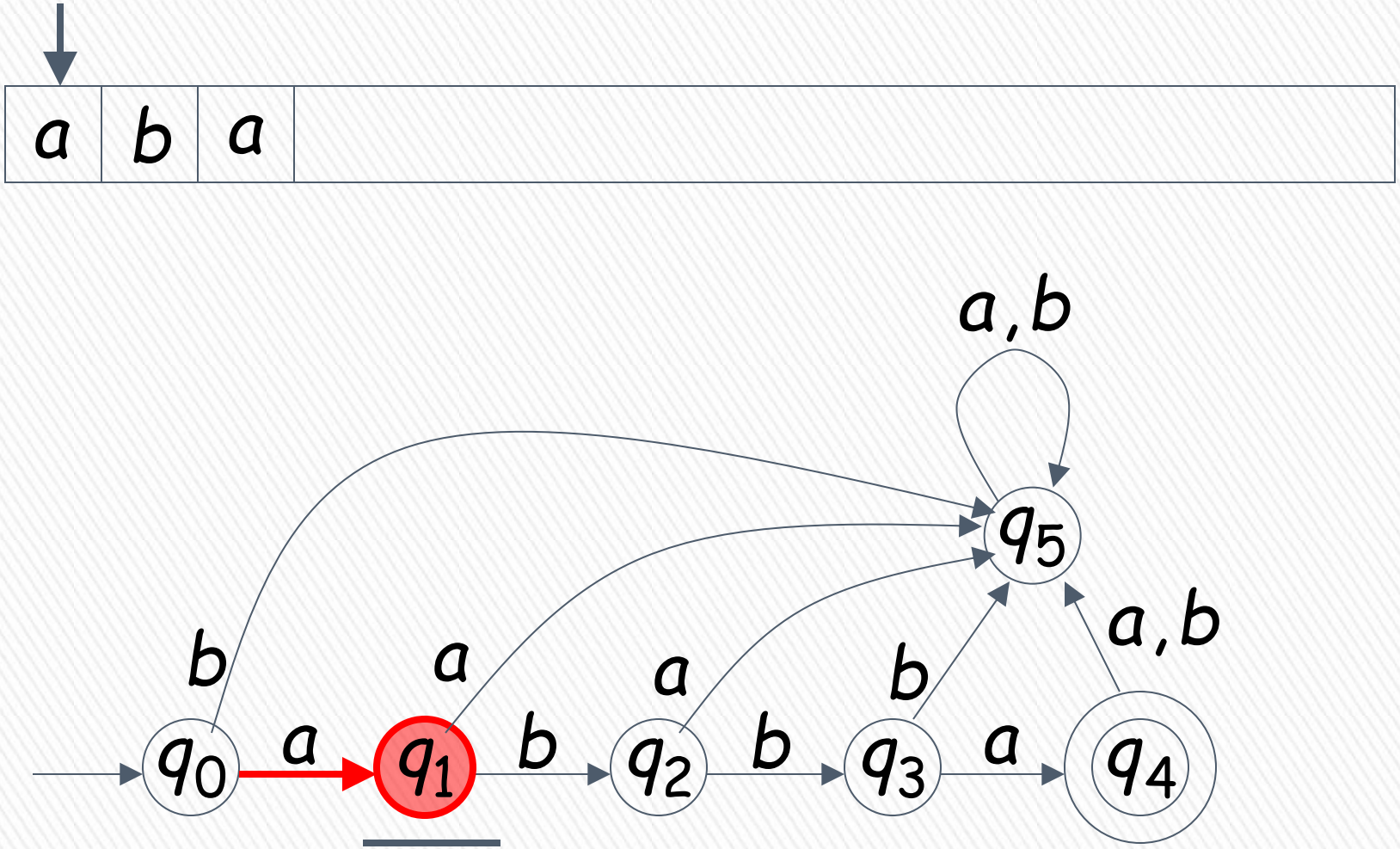
BLM2502 Theory of Computation

Input String

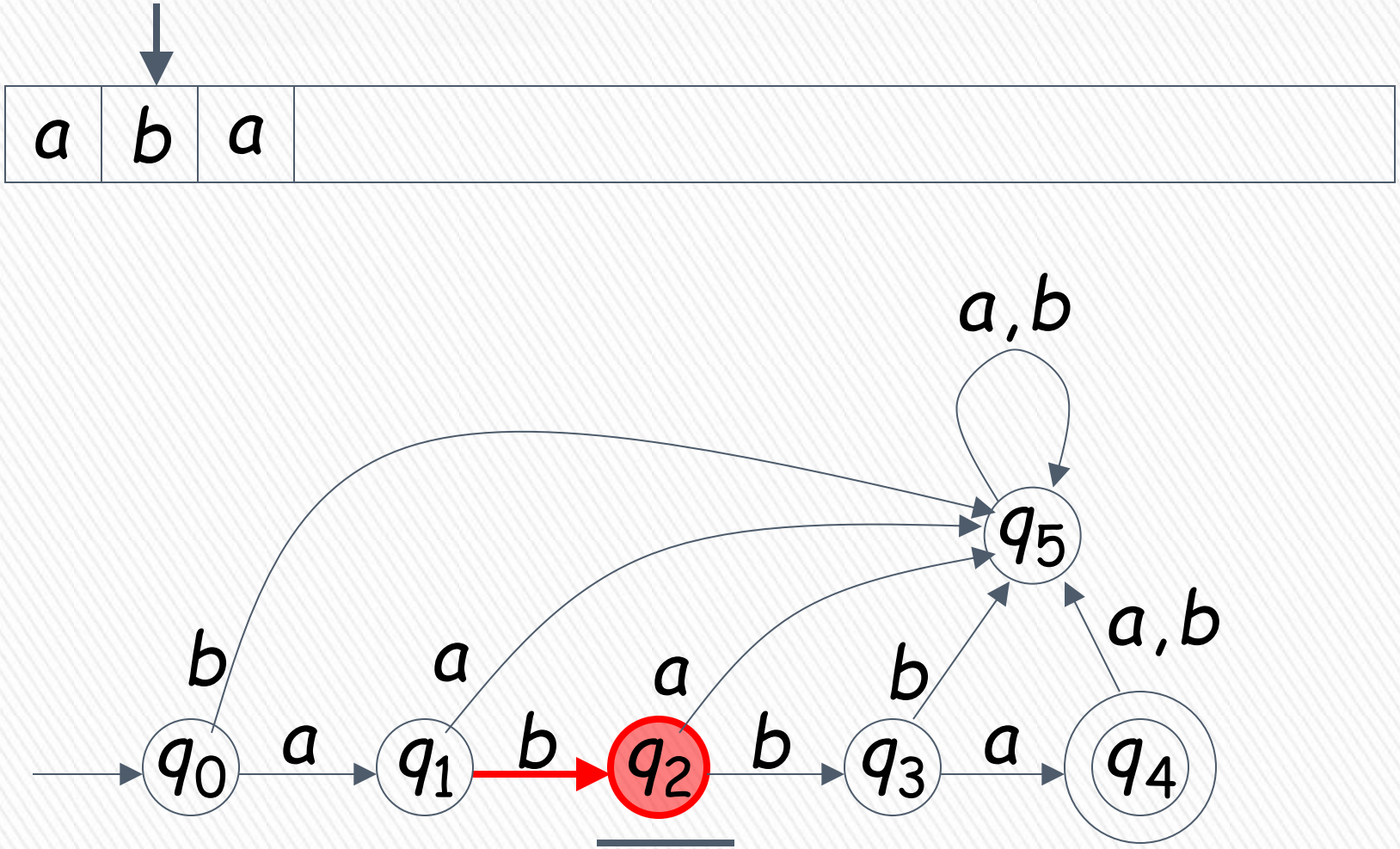


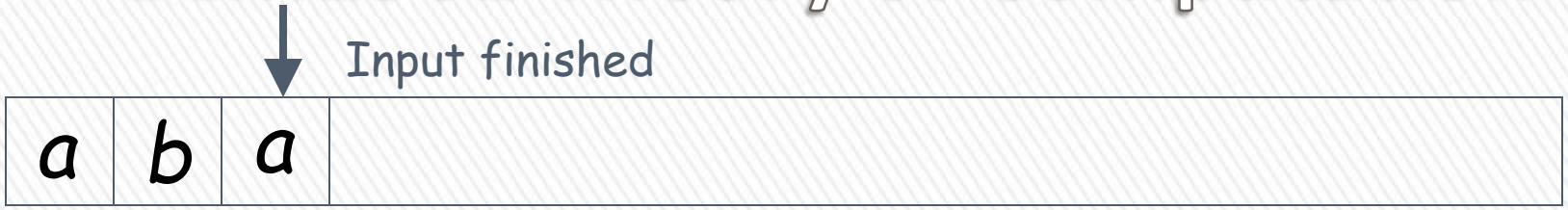
A Rejection Case

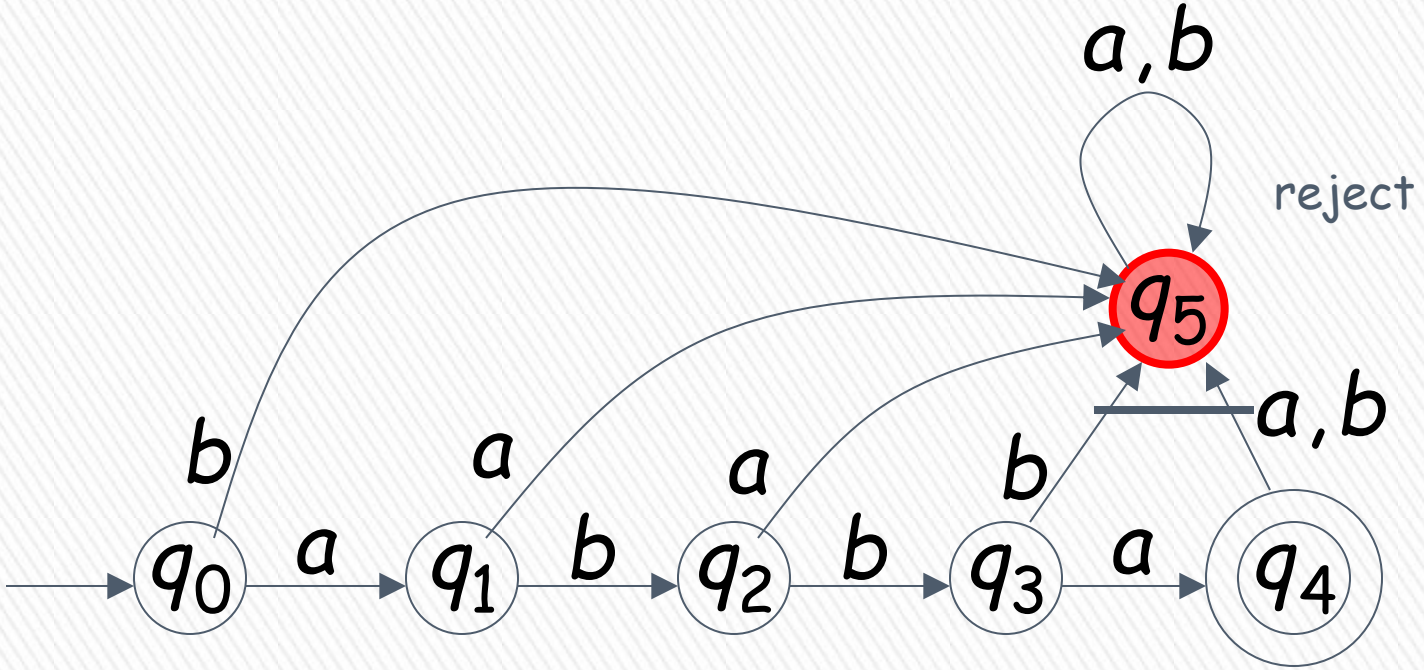
BLM2502 Theory of Computation



BLM2502 Theory of Computation



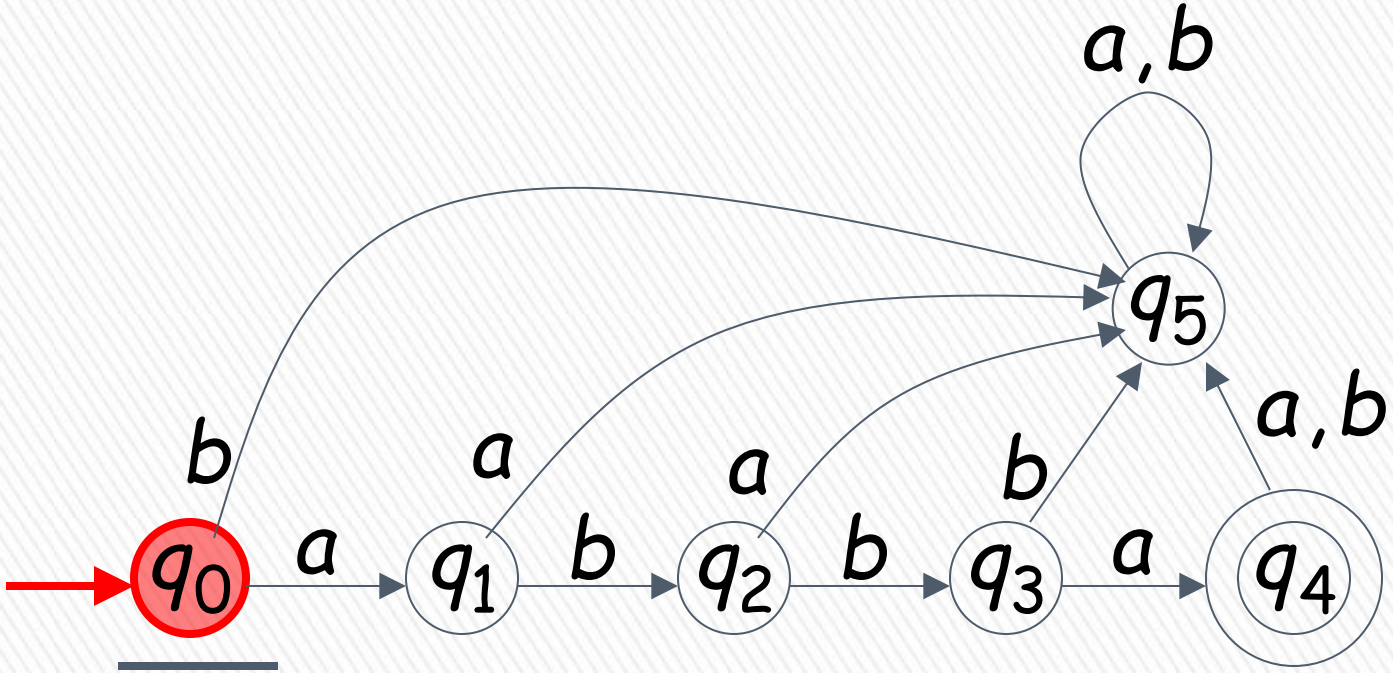

Input finished
a
b
a


a,b
reject
q5
a,b
b
a
a
b
q0
a
q1
b
q2
b
q3
a
q4

BLM2502 Theory of Computation

Tape is empty

Input Finished



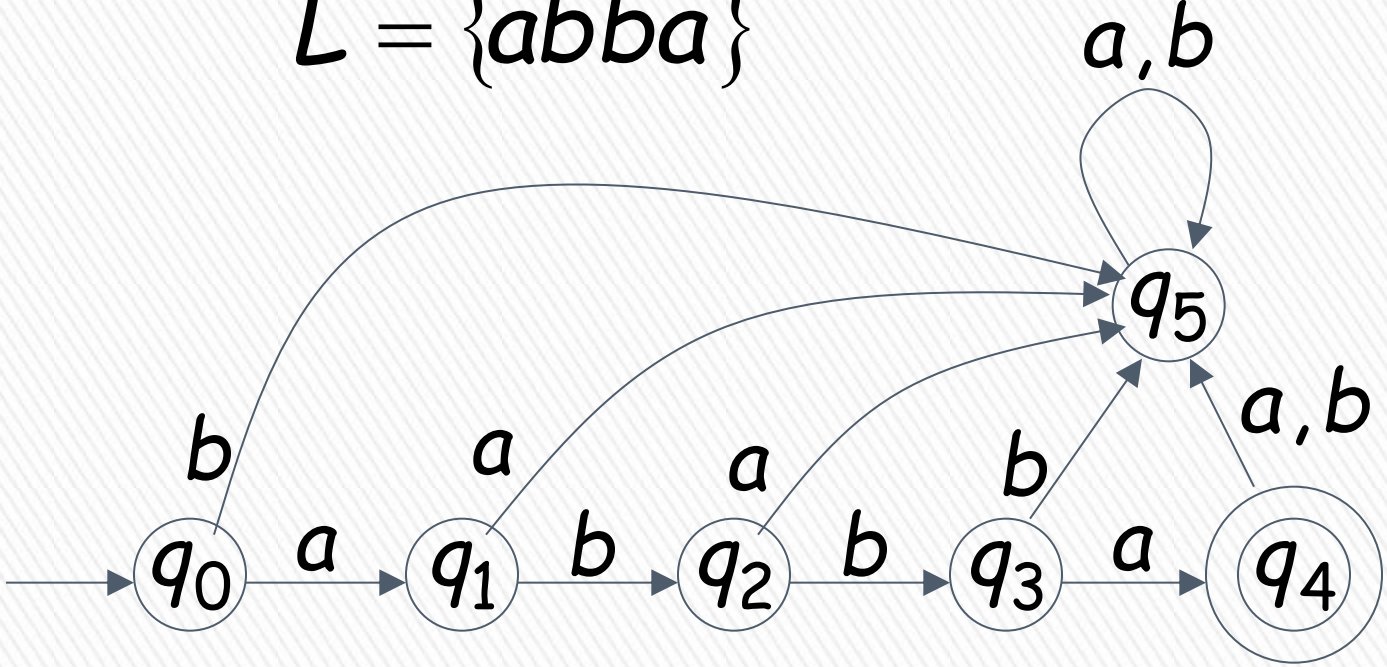
reject

Another Rejection Case

BLM2502 Theory of Computation

Language Accepted:

$$L = \{abba\}$$



BLM2502 Theory of Computation

To accept a string:

- all the input string is scanned

and

- the last state is accepting

To reject a string:

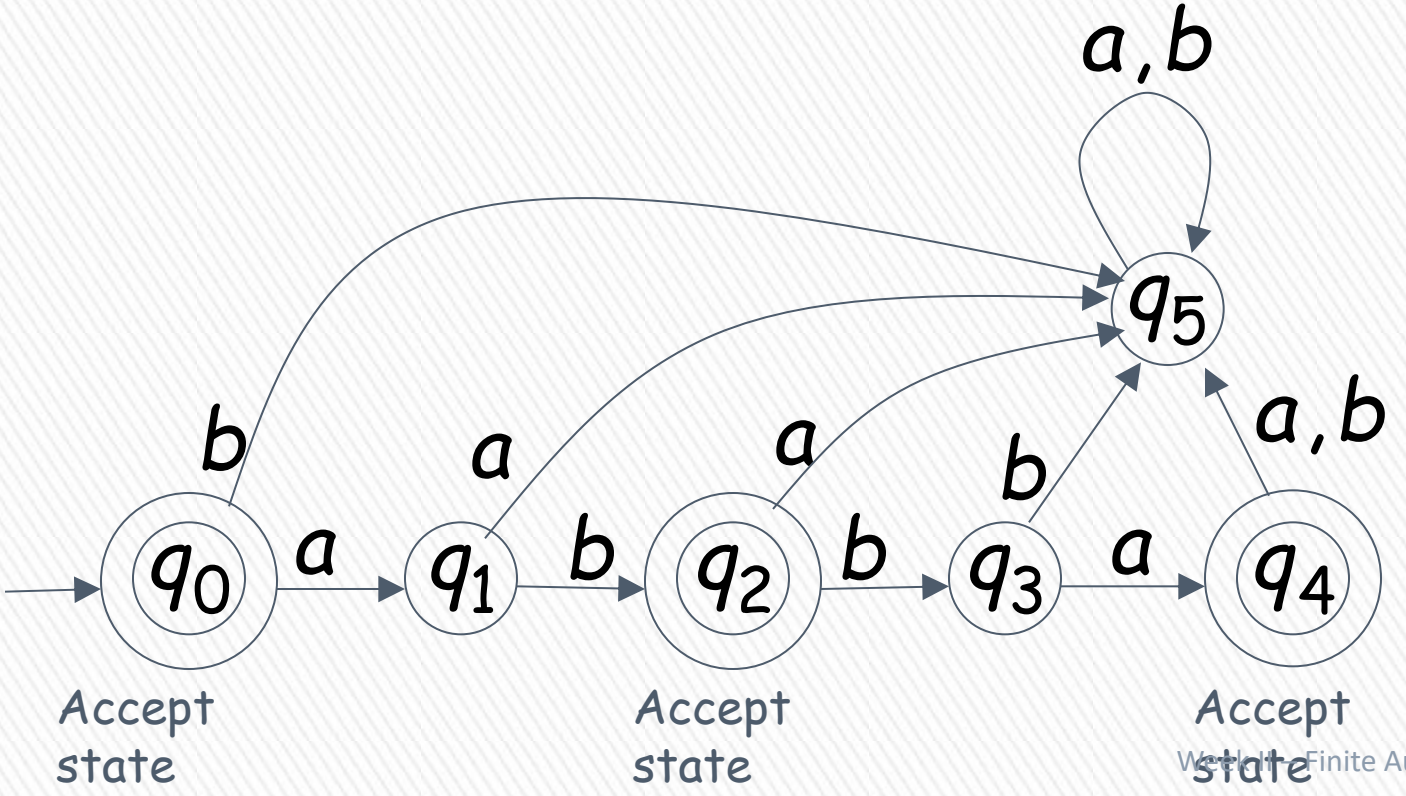
- all the input string is scanned

and

- the last state is non-accepting

BLM2502 Theory of Computation

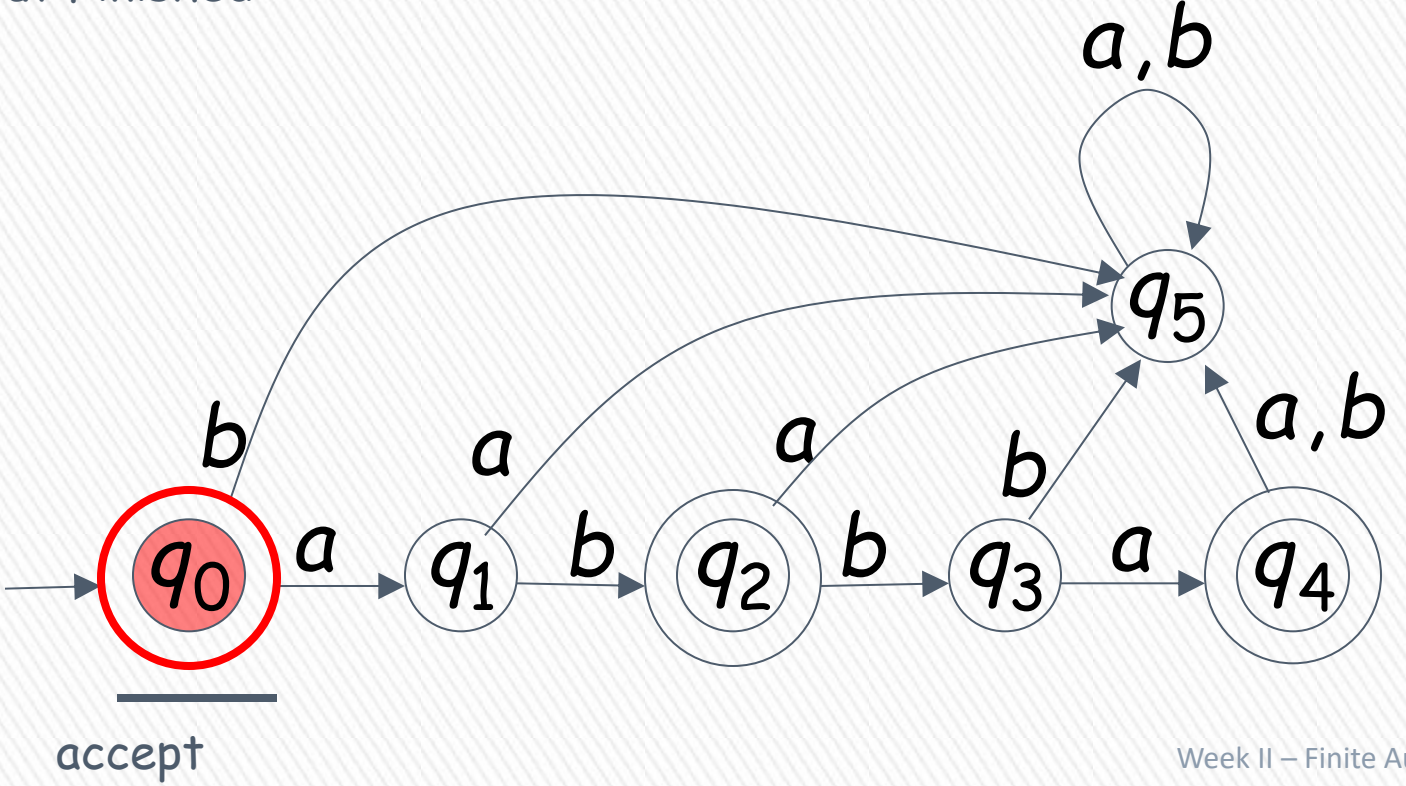
L = {ε, ab, abba}



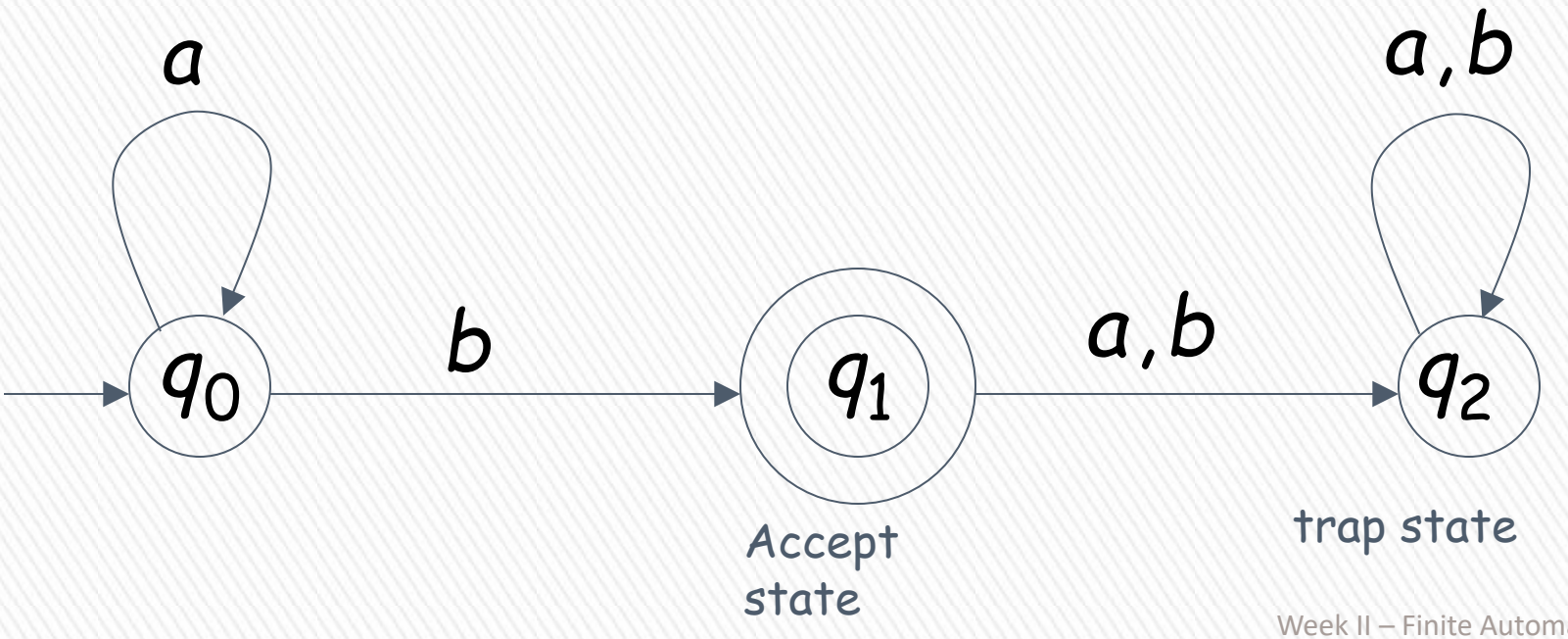
BLM2502 Theory of Computation

Empty Tape

Input Finished

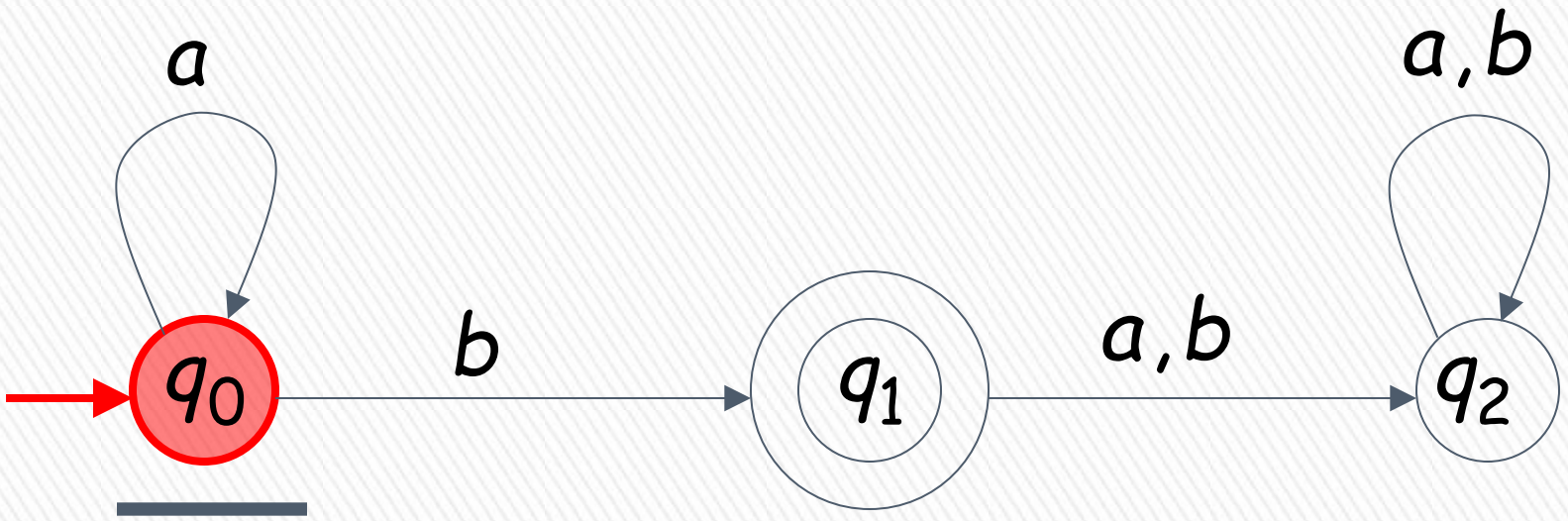


BLM2502 Theory of Computation

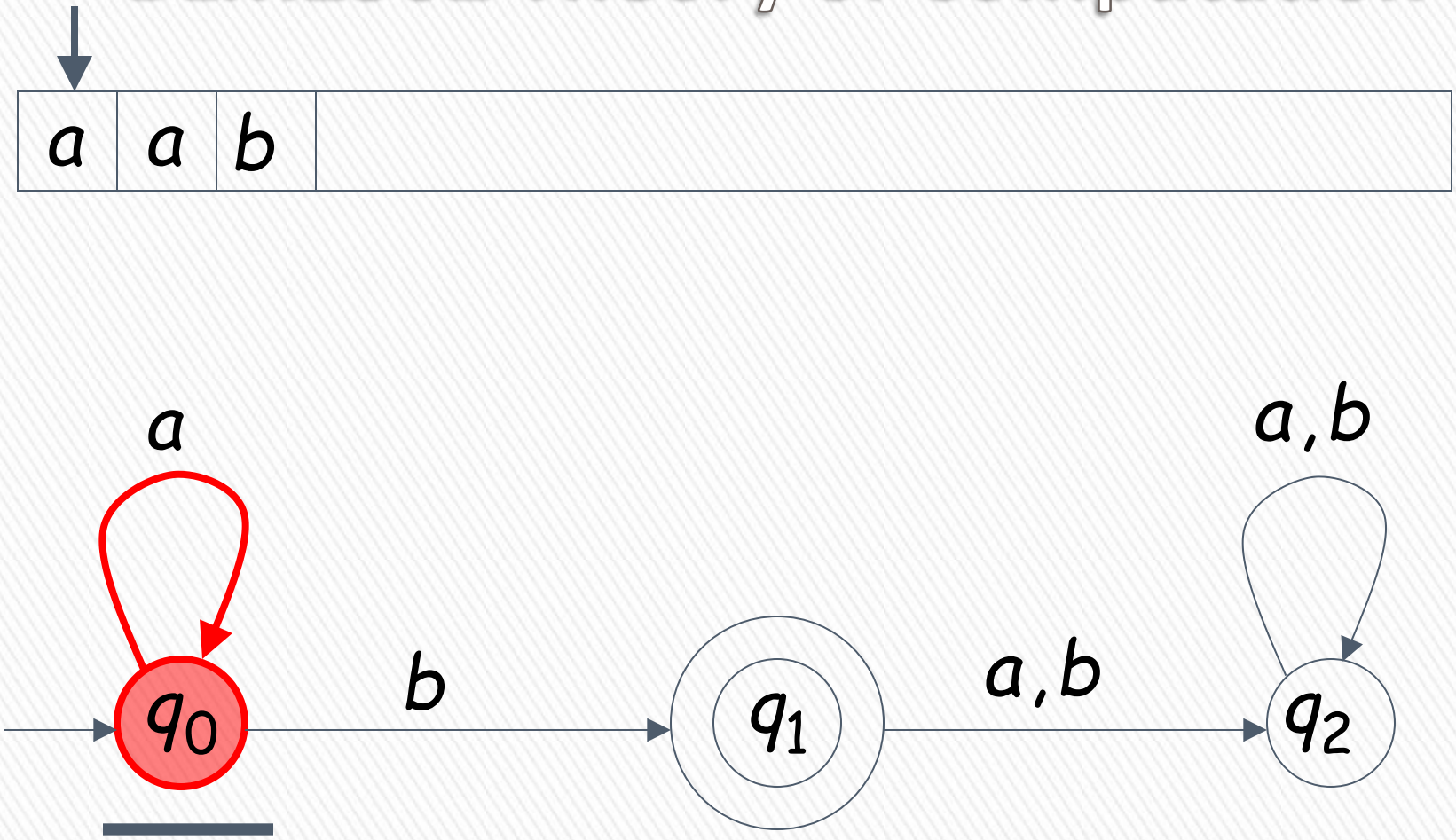


BLM2502 Theory of Computation

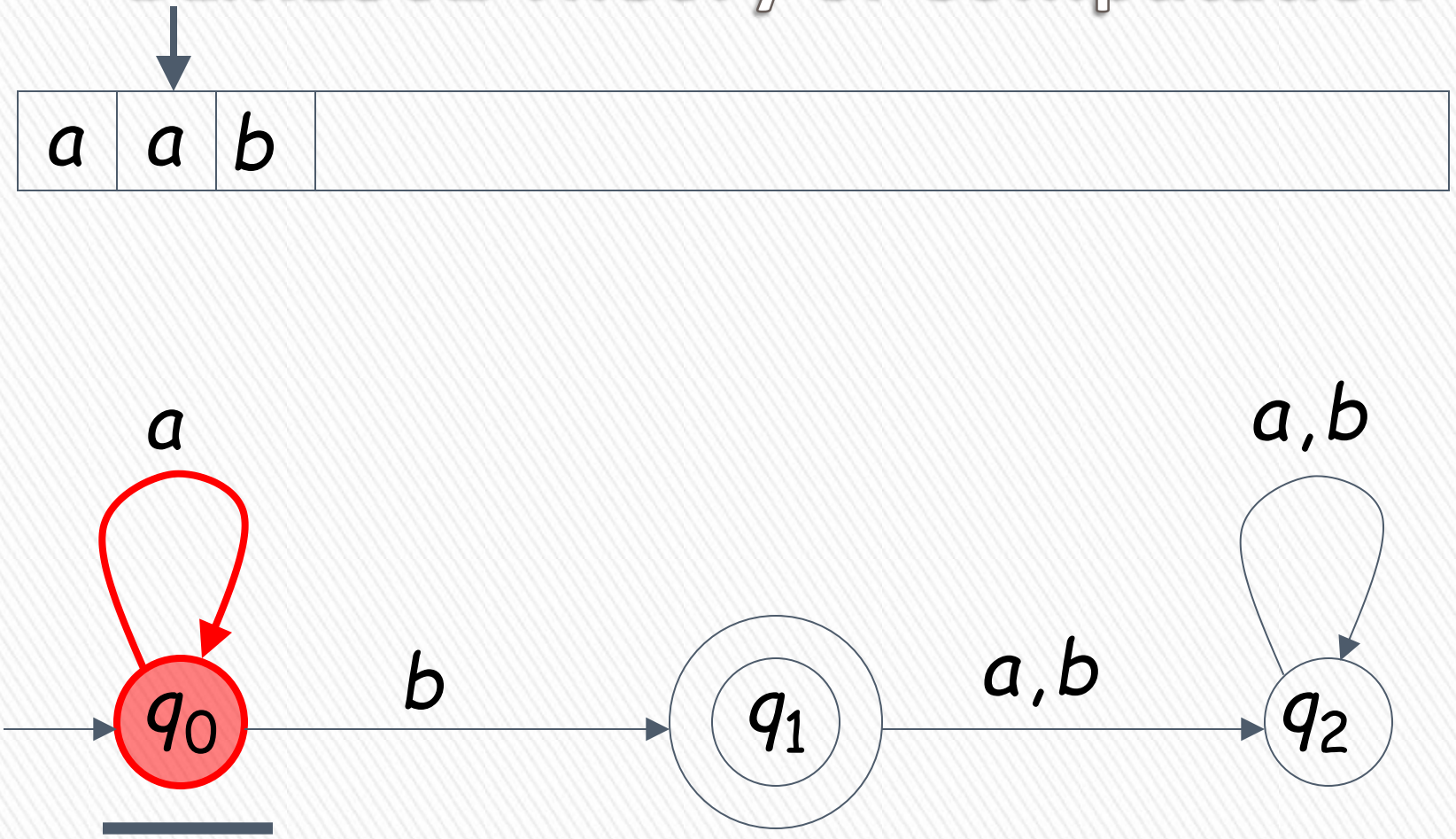
Input String

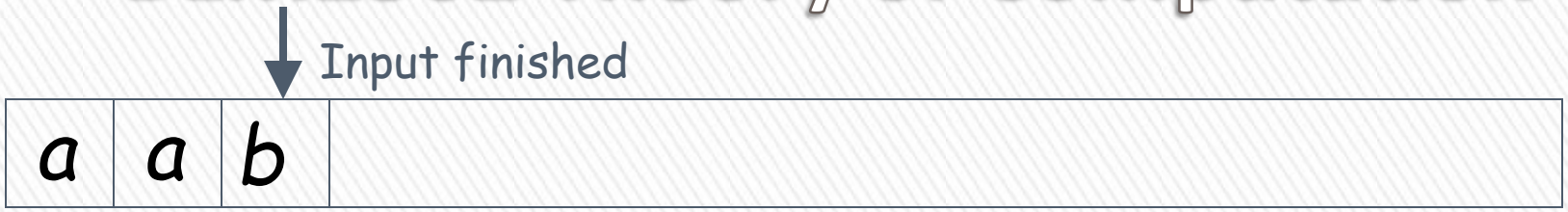


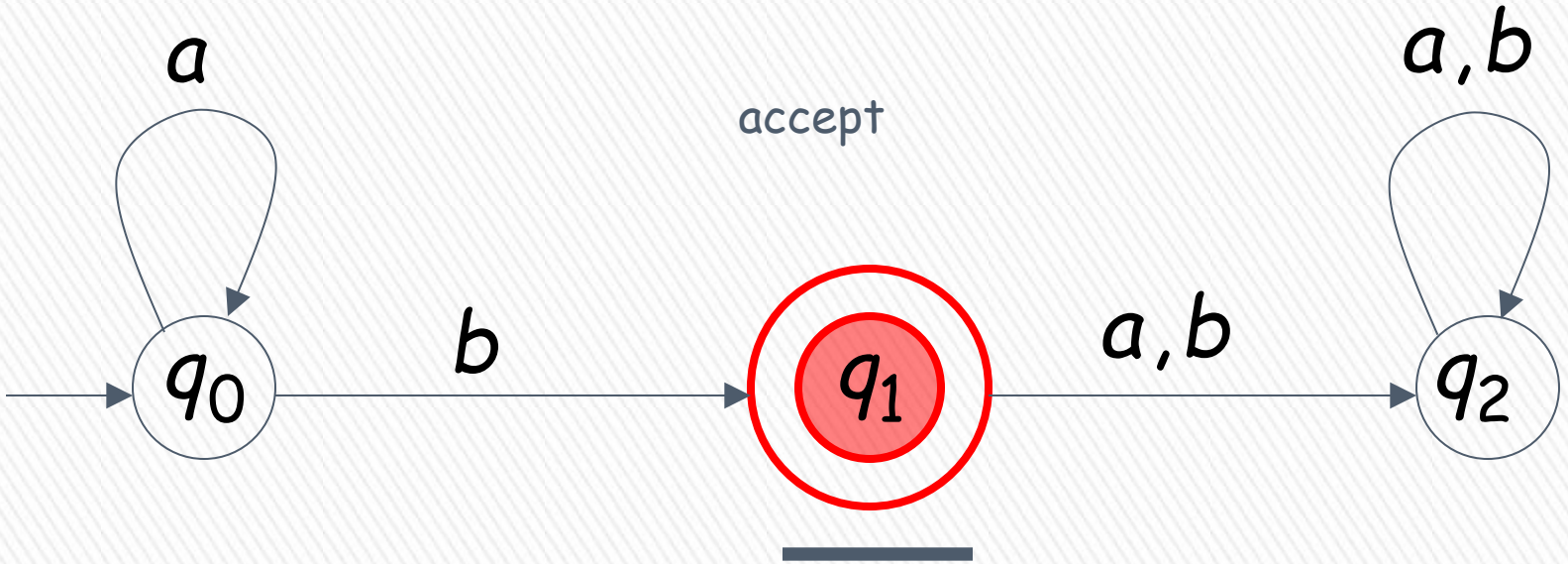
BLM2502 Theory of Computation



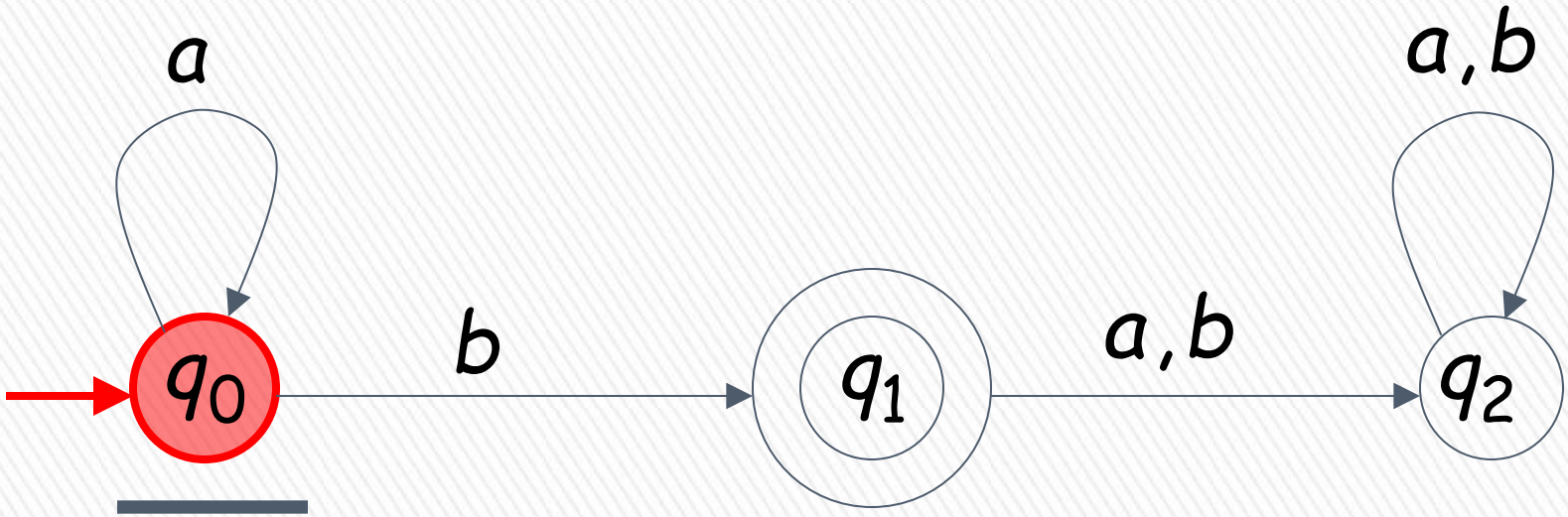
BLM2502 Theory of Computation




Input finished
a
a
b

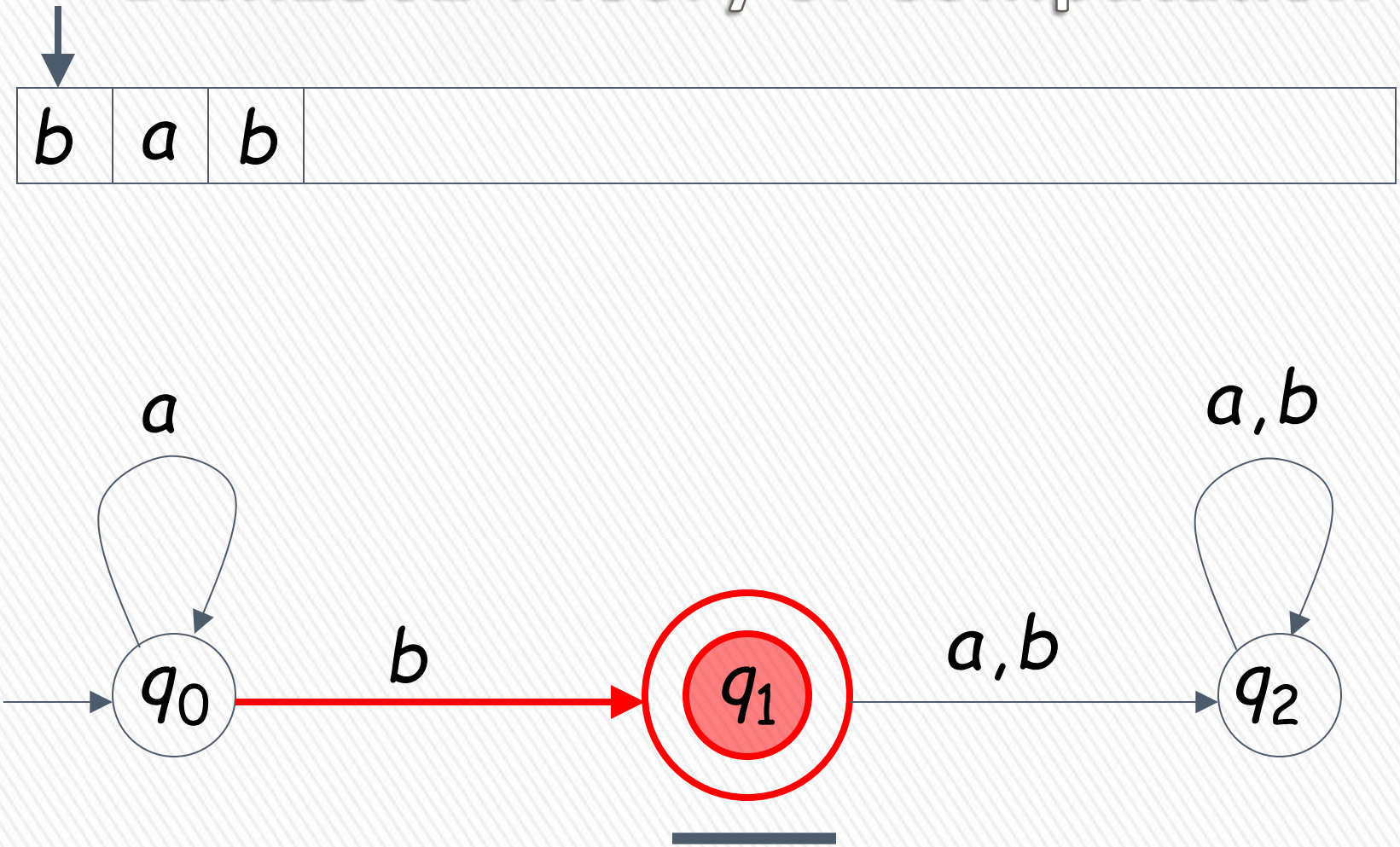

a
accept
a, b
b
a, b
q_0
q_1
q_2

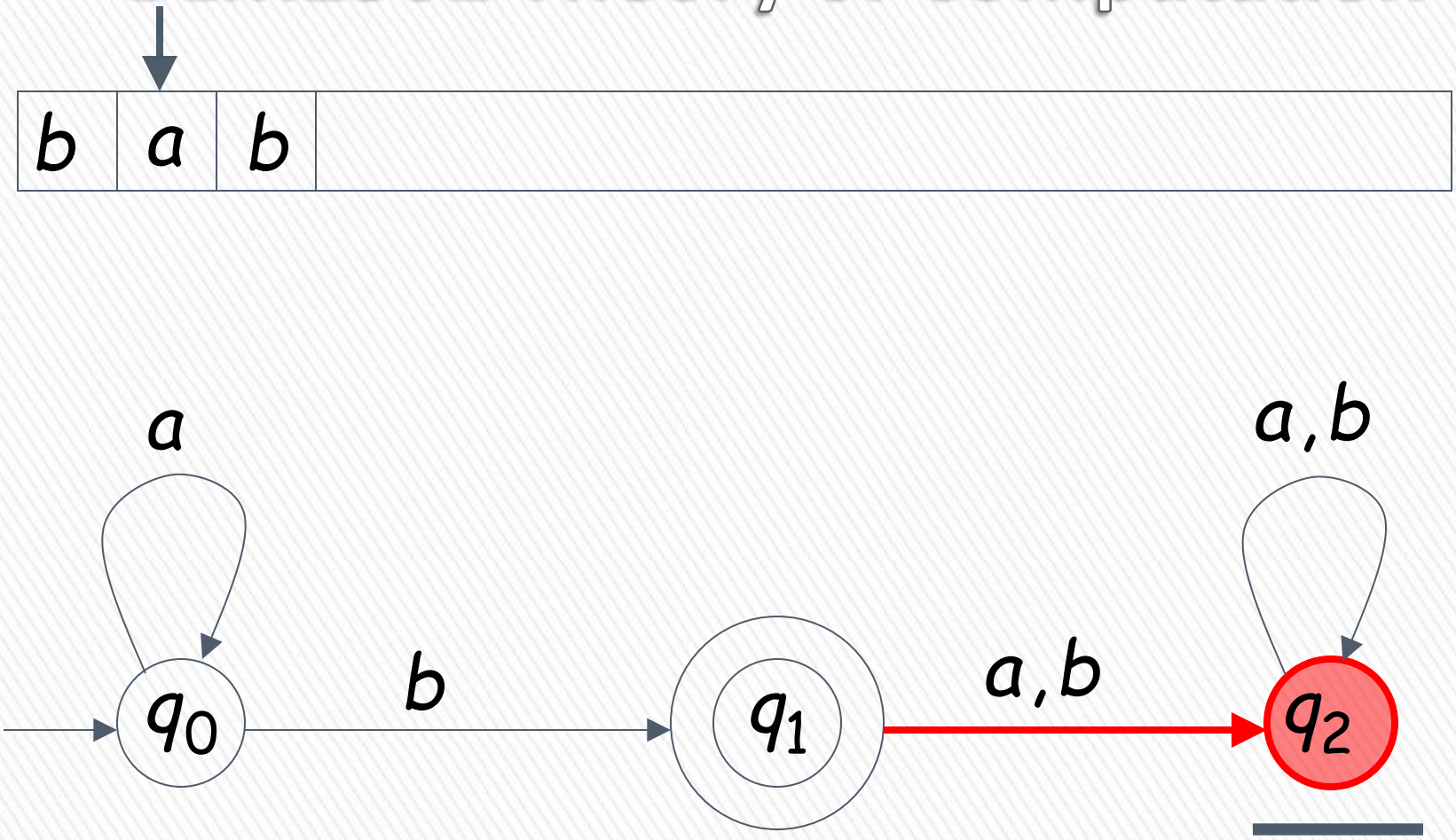
Input String

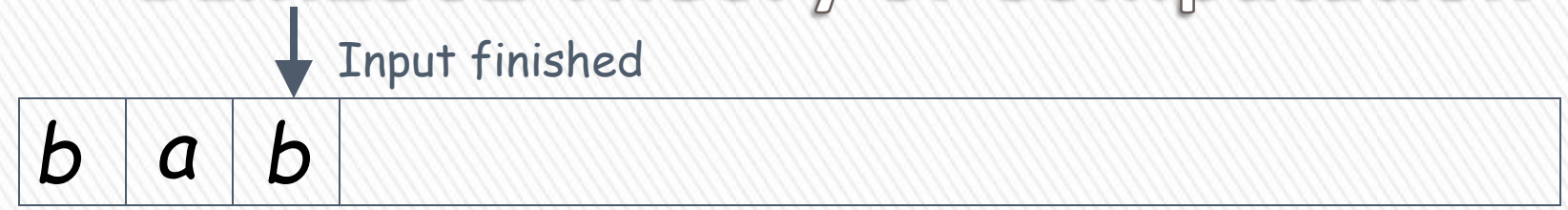


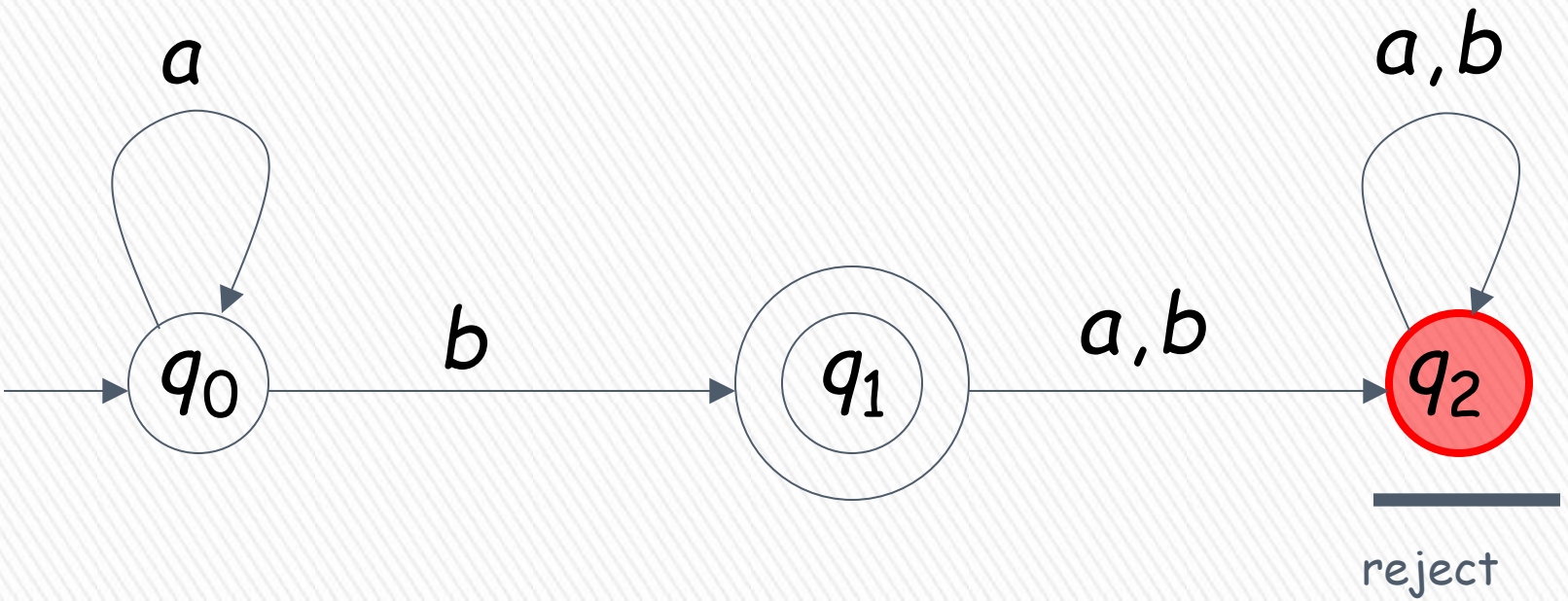
A rejection case

BLM2502 Theory of Computation




b
a
b
a
a, b
a, b
b
q_0
q_1
q_2

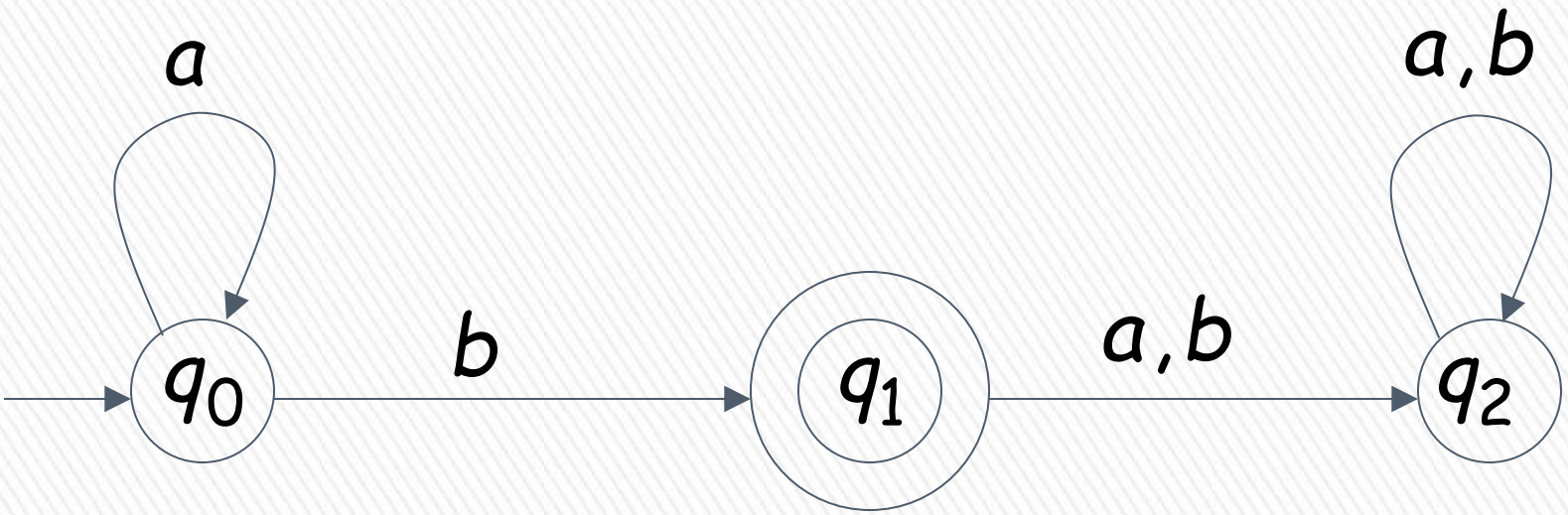

Input finished
b
a
b


a
a,b
b
a,b
q_0
q_1
q_2
reject

BLM2502 Theory of Computation

Language Accepted:

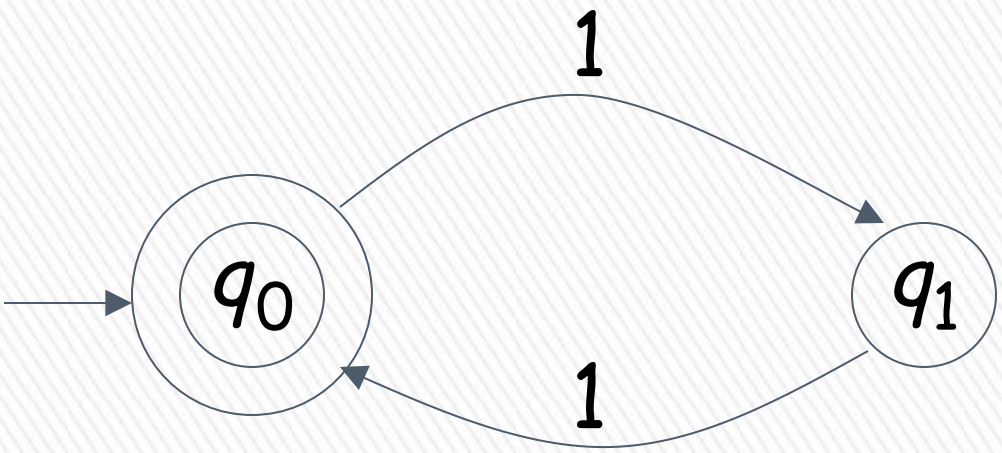
$$L = \{a^n b : n \geq 0\}$$



BLM2502 Theory of Computation

$L = \{1^n : n = 2k, k \text{ is integer}\}$

Alphabet: $\Sigma = \{1\}$



Language Accepted:

EVEN = {x: x is in Σ^* and x is even}

= {ε, 11, 1111, 111111, … }

BLM2502 Theory of Computation

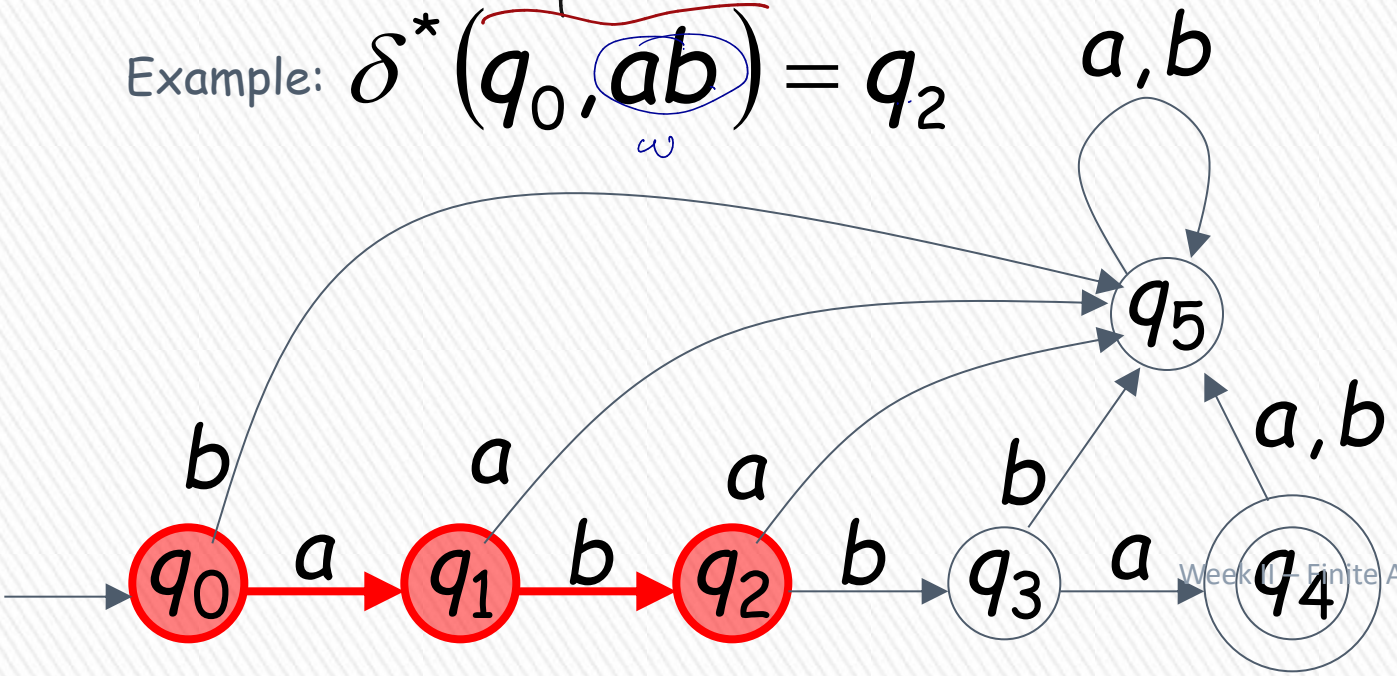
» Extended Transition Function

$\delta^* : Q \times \Sigma^* \rightarrow Q$

$\delta(q, w) \rightarrow q'$

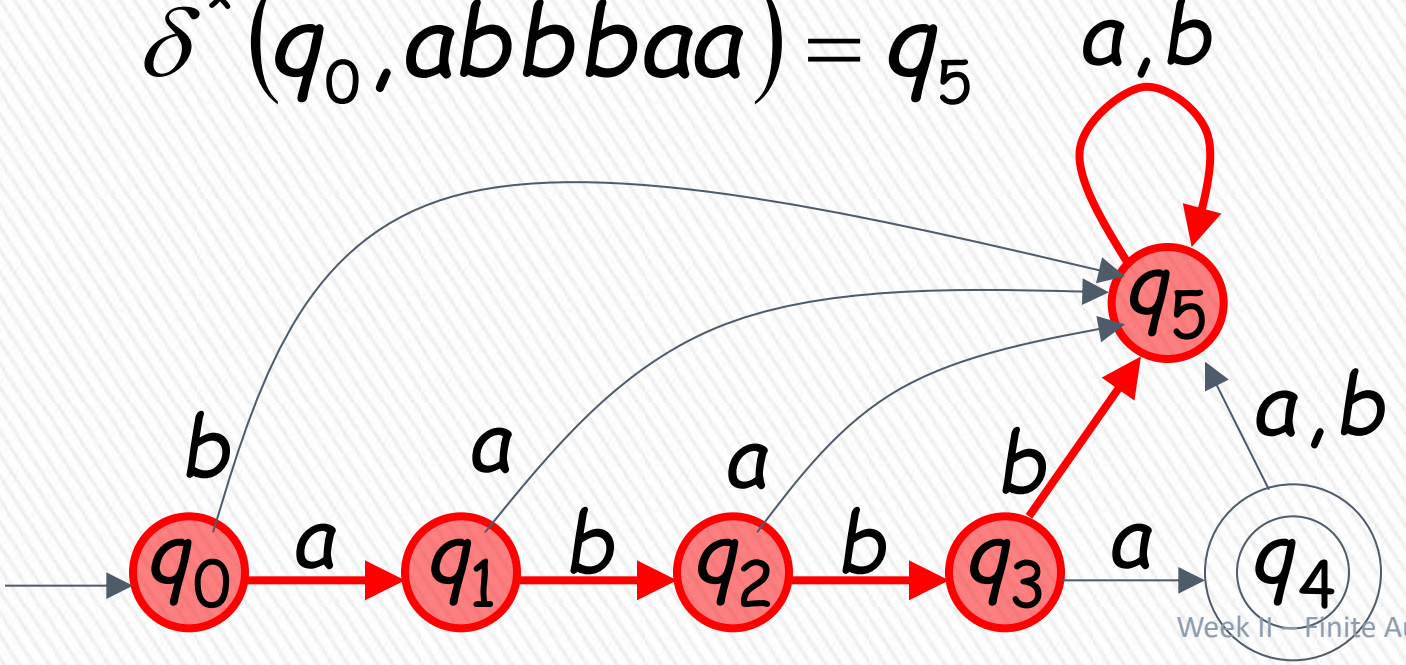
> Describes the resulting state after scanning string **w** from state **q**

Example: $\delta^*(q_0, ab) = q_2$



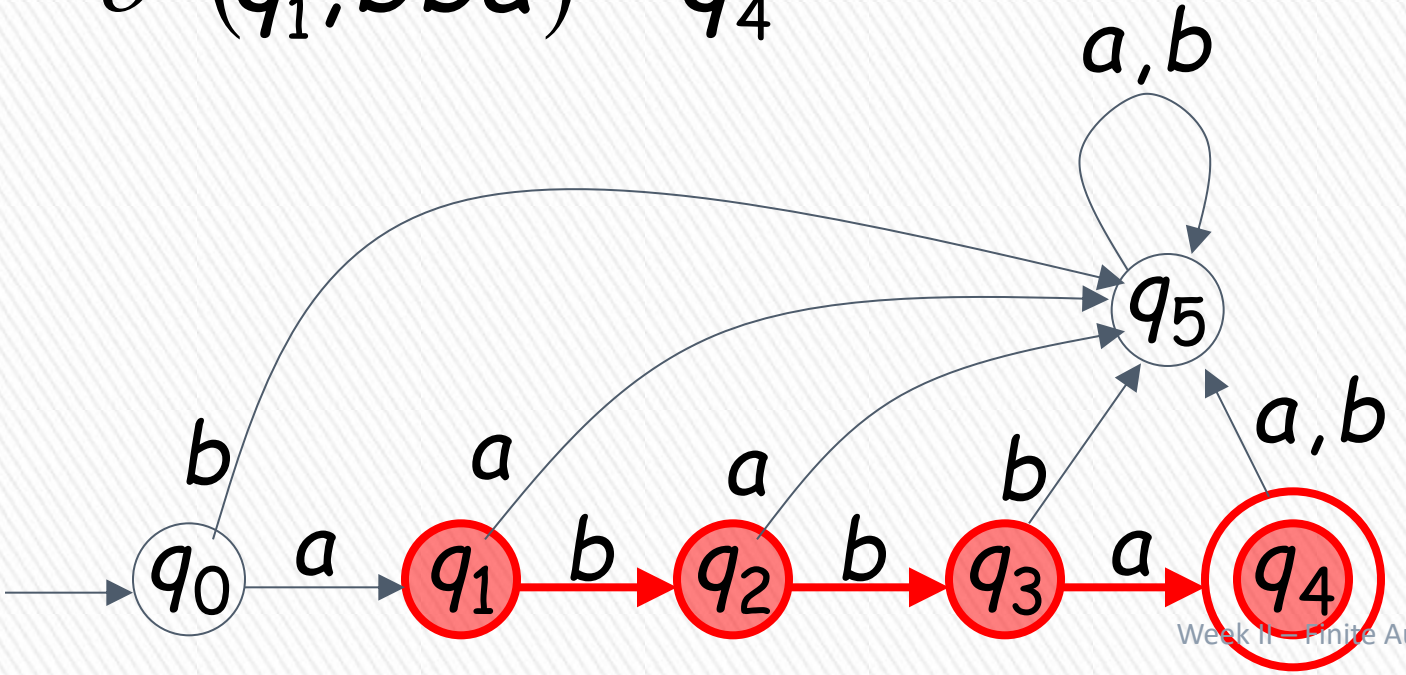
BLM2502 Theory of Computation

$$\delta^*(q_0, abbbaa) = q_5$$



BLM2502 Theory of Computation

$$\delta^*(q_1, bba) = q_4$$



BLM2502 Theory of Computation

In general:

δ* (q, w) →q' implies that there is a walk of transitions

$$w = \sigma_1 \sigma_2 \cdots \sigma_k$$

states may be repeated

Language Accepted By DFA

The language accepted by an automaton M, is denoted as L(M) and contains all the strings accepted by M

We say that a language L' is accepted (or recognized) by the DFA M if

$$L(M) = L'$$

An automaton accepts one and only one language.
A language can be accepted by a number of automata

BLM2502 Theory of Computation

- For a DFA **M** = (Q, Σ, δ, q_0, F)
- Language accepted by **M**:

$$L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \in F\}$$

BLM2502 Theory of Computation

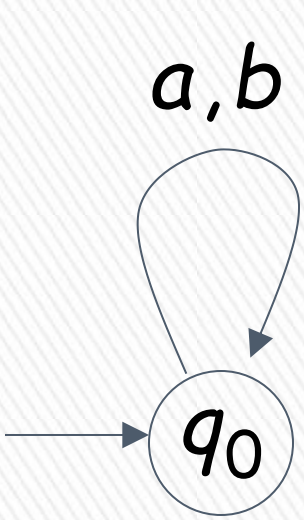
» Language rejected by **M** :

$$\overline{L(M)} = \{w \in \Sigma^* : \delta^*(q_0, w) \notin F\}$$

BLM2502 Theory of Computation

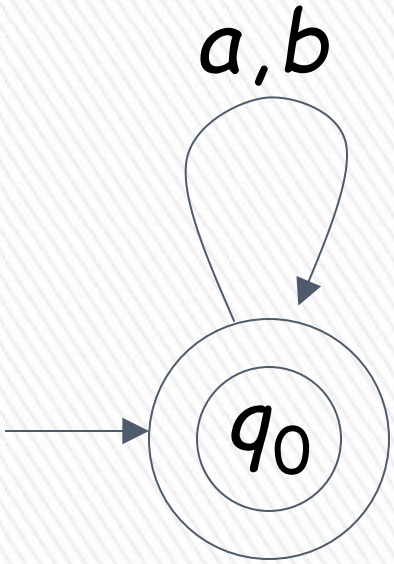
More DFA Examples

$\Sigma = \{a, b\}$



$L(M) = \{\ \}$

Empty language

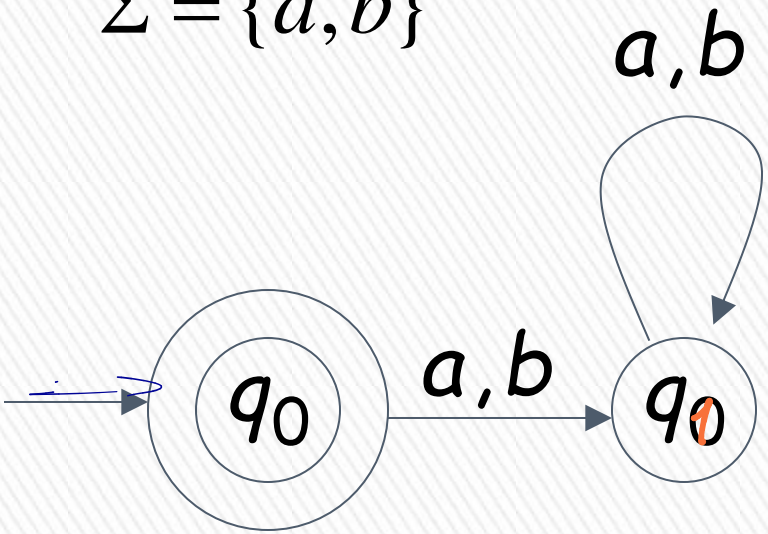


$L(M) = \Sigma^*$

All strings

BLM2502 Theory of Computation

$$\Sigma = \{a, b\}$$



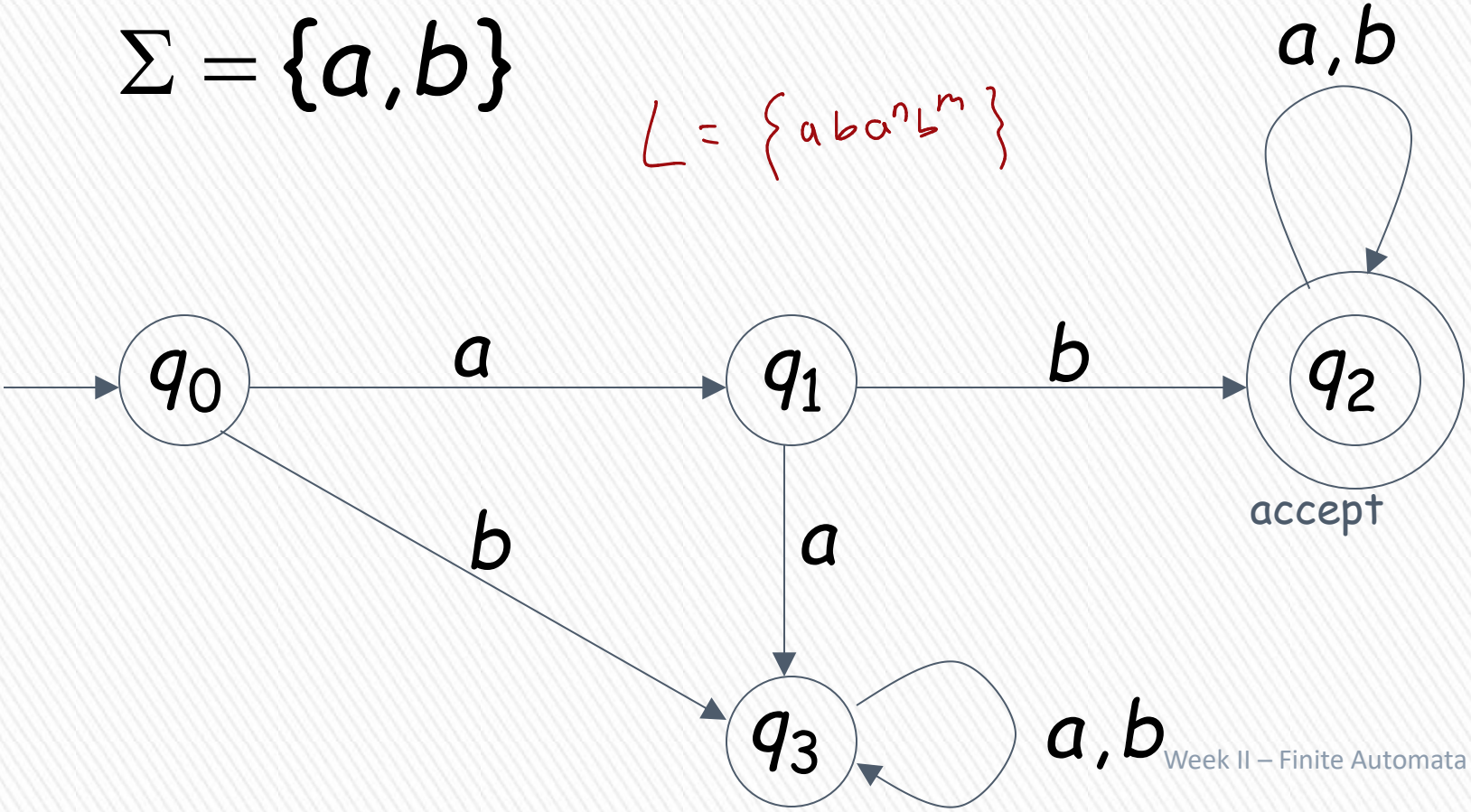
$$L(M) = \{\lambda\}$$

Language of the empty string

L(M) = { all strings with prefix ab }

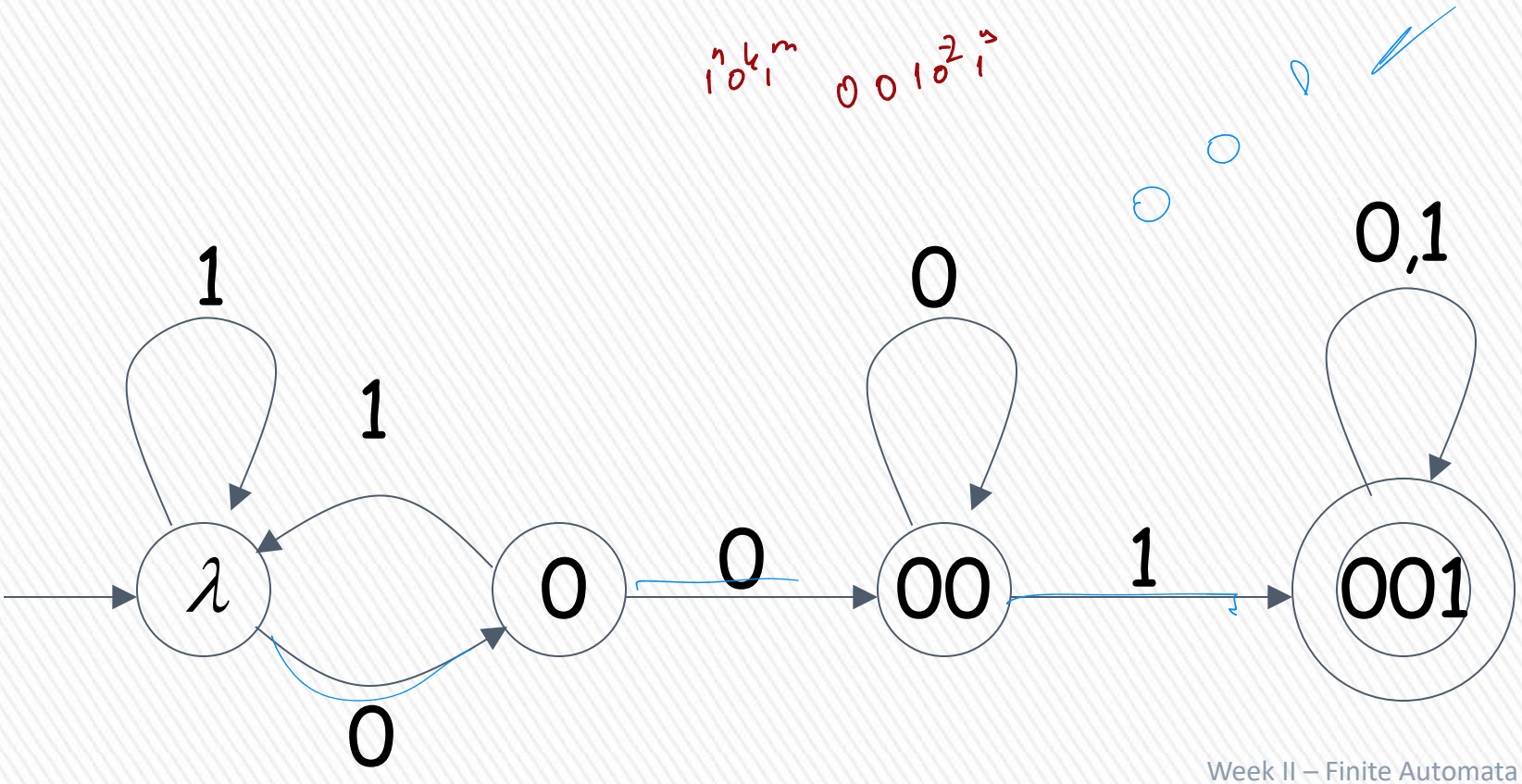
$\Sigma = \{a, b\}$

$L = \{ab a^n b^m\}$



BLM2502 Theory of Computation

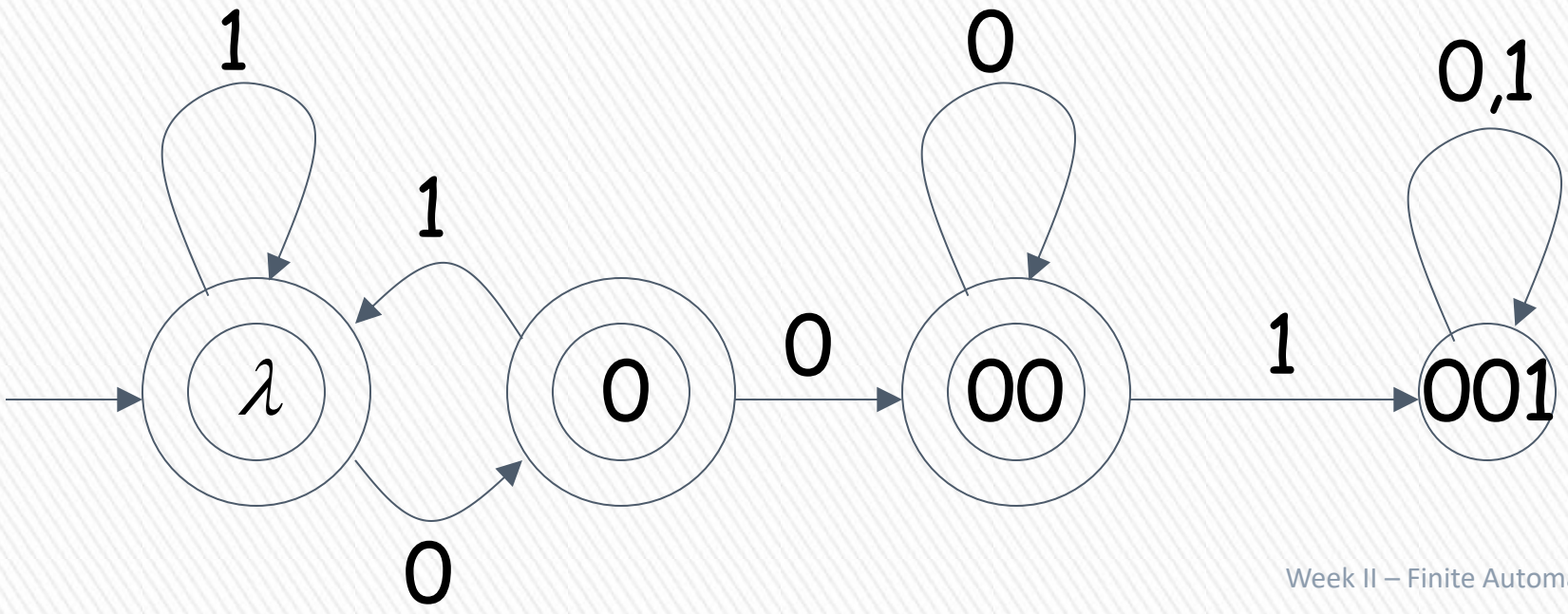
L(M) = { all binary strings containing substring 001 }



BLM2502 Theory of Computation

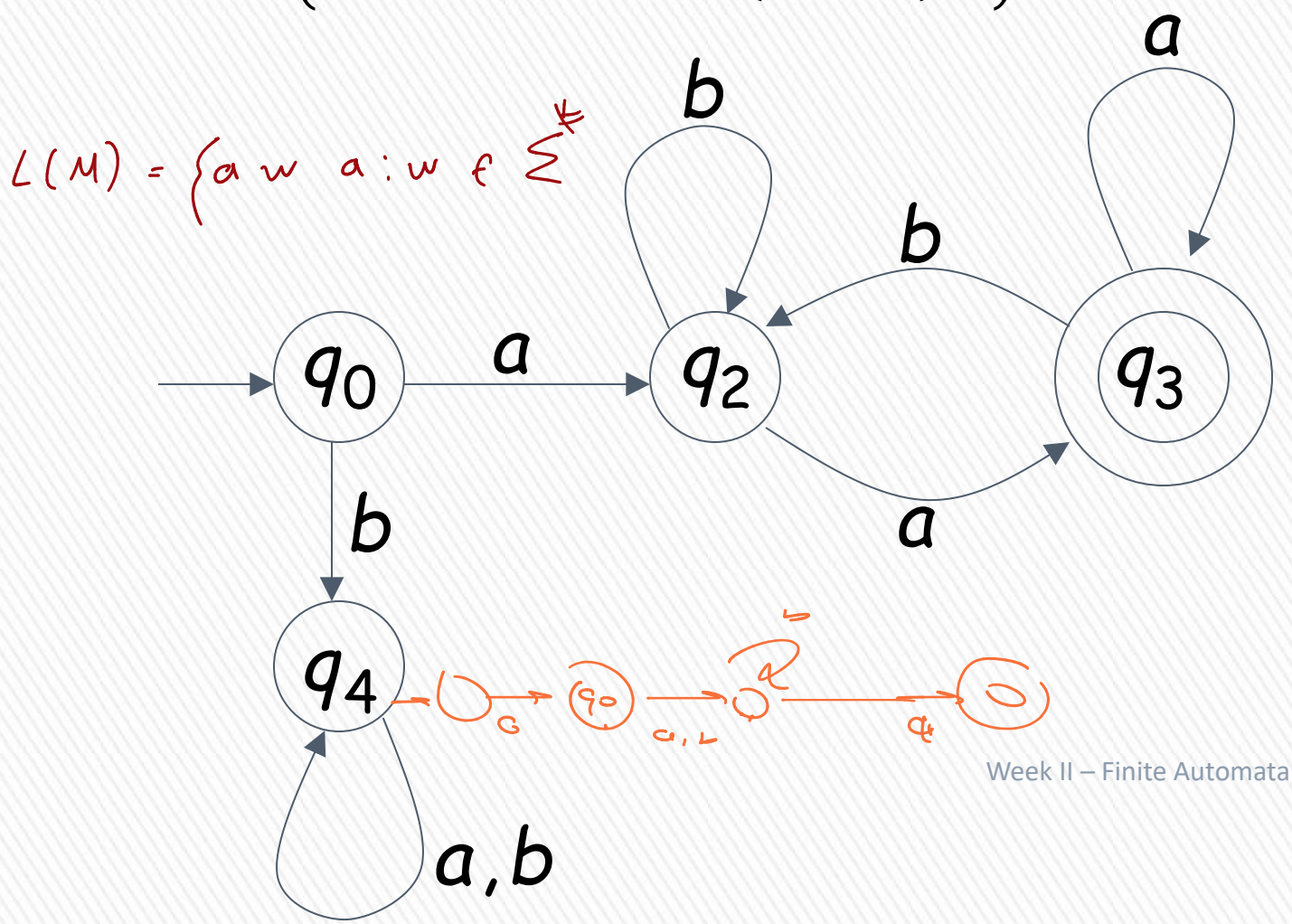
L(M) = { all binary strings without substring 001}

all binary strings without substring 001



BLM2502 Theory of Computation

$$L(M) = \{awa : w \in \{a,b\}^*\}$$



BLM2502 Theory of Computation

$\{abba\}$ $\{\lambda, ab, abba\}$

$\{a^n b : n \geq 0\}$ $\{awa : w \in \{a,b\}^*\}$

$\{x : x \in \{1\}^*$ and x is even$\}$

$\{\}$ $\{\lambda\}$ $\{a,b\}^*$

{ all binary strings without substring 001 }

{ all strings in {a,b}* with prefix ab }

There exist automata that accept these languages (see previous slides).

BLM2502 Theory of Computation

There exist languages which are not Regular:

$$L = \{a^n b^n : n \geq 0\}$$

$$ADDITION = \{x + y = z \ : \mathrm{x} = 1^{\mathrm{n}}, \mathrm{y} = 1^{\mathrm{m}},$$

$$\mathrm{z} = 1^{k}, \ \mathrm{n} + m = k\}$$

There is no DFA that accepts these languages